



Natural Language Processing

Session 1

Agenda

1 Natural Language Processing Introduction

2 Preprocessing in NLP

3 Syntactic Analysis

4 Semantic Analysis

Natural Language Processing Introduction

Human Language

HUMANS



OTHER SPECIES



Human Language



LANGUAGE



ALPHABETS

字母 वर्णमाला ALFABETOS
الأبجدية எழுத்துக்கள்



This is a Sentence

Human Language

**KNOW THE
RULES !**

LANGUAGE



ALPHABETS

! 字母 वर्णमाला ALFABETOS
الأبجدية எழுத்துக்கள்



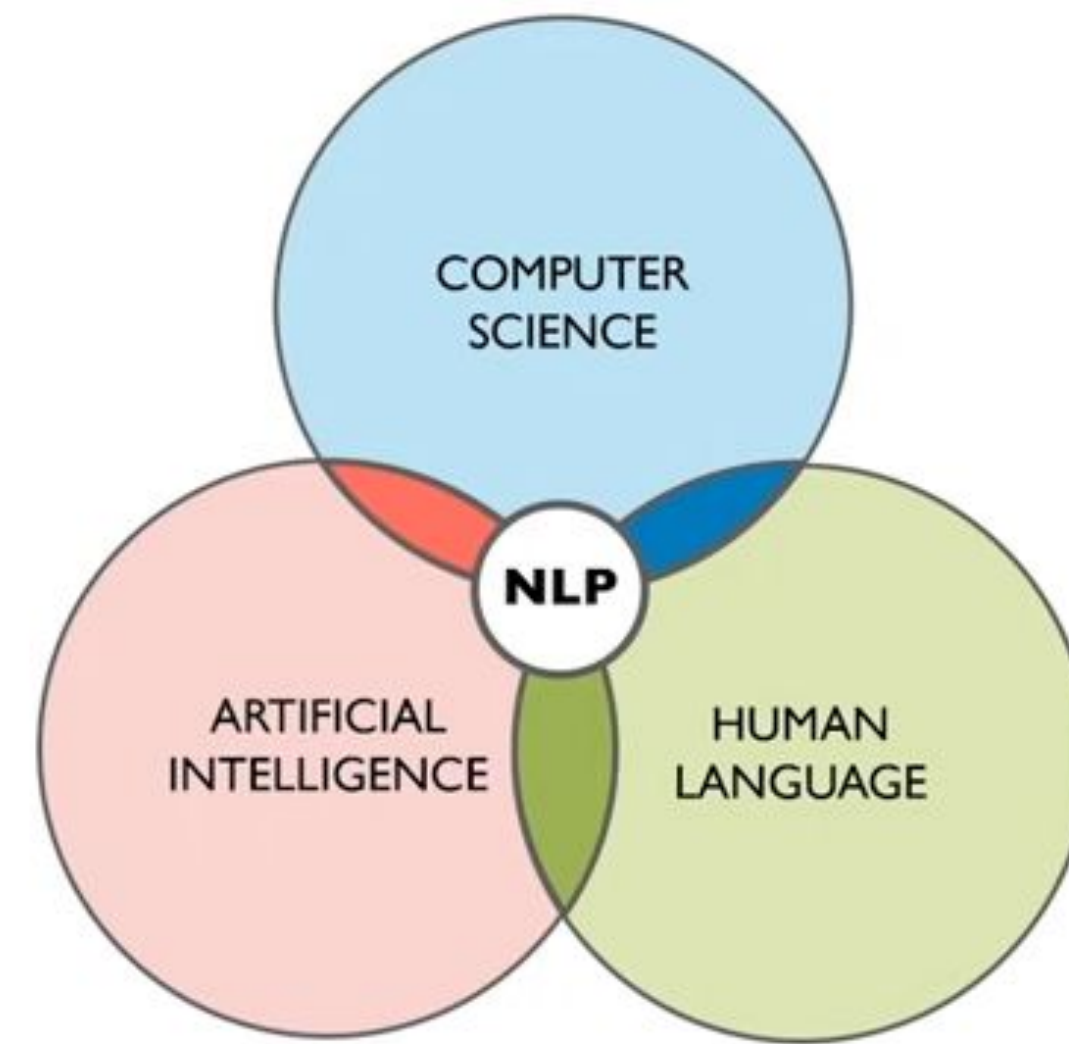
This is a Sentence

infinitives
tenses
adjectives
relative
future
affirmative
negative
exercises
adverbs
interrogatives
present
articles
nouns
past
negative
conditional
worksheets
english
gerunds
questions
prepositions
passives
reported
verbs

Natural Language Processing



NLP: Natural Language Processing is a part of computer science and artificial intelligence which deals with human languages.



Applications of NLP



Spell
Checking

Information
Extraction



Advertisement
Matching



Keyword
Search



Sentimental
Analysis

Chatbot

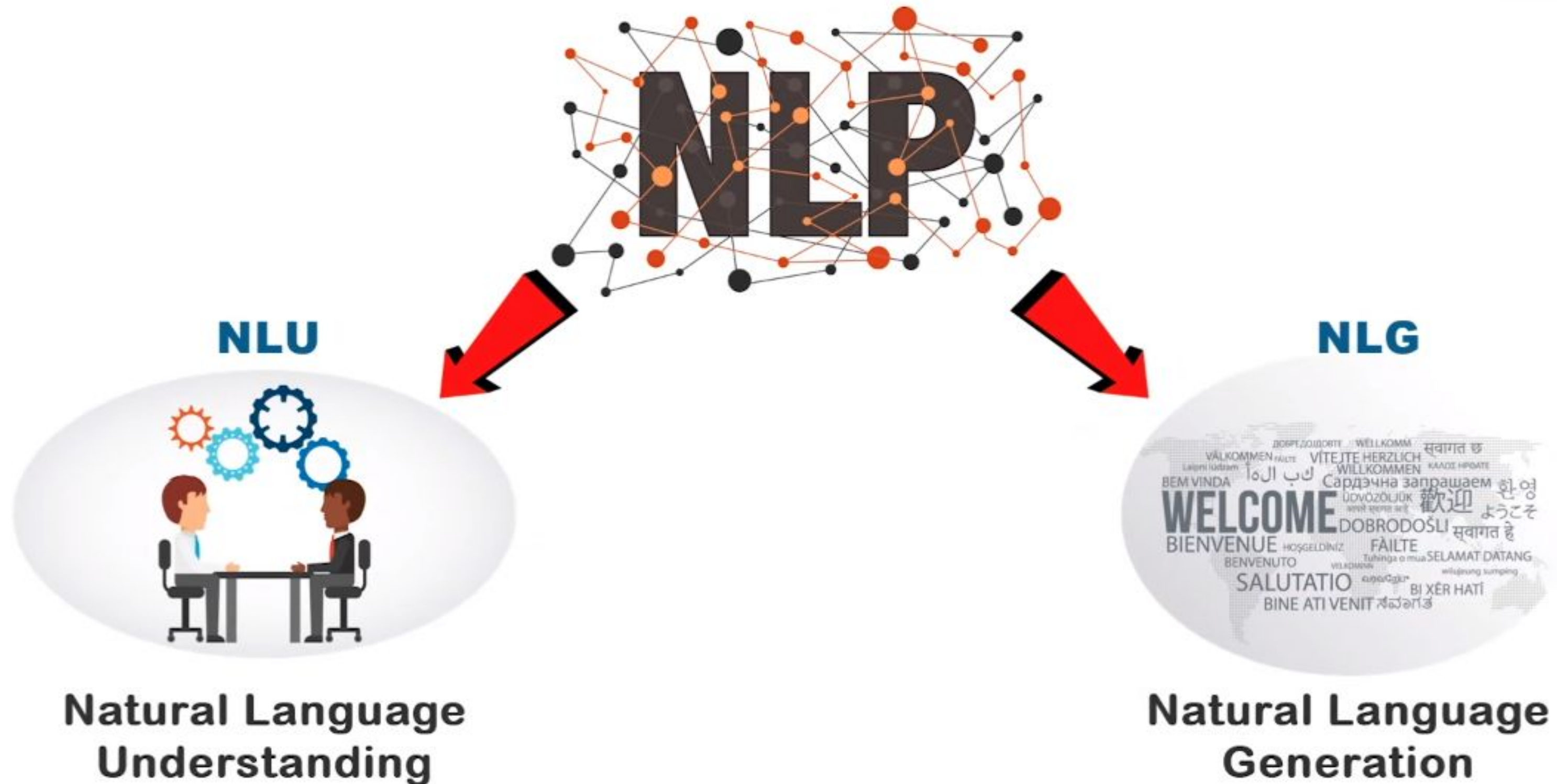


Speech
Recognition

Machine
Translation



NLP Components



NLP Components



Natural Language Understanding

- **Mapping input to useful representations**
- **Analyzing different aspects of the language**

- **Text Planning**
- **Sentence Planning**
- **Text Realization**



Natural Language Generation

NLU Ambiguity

Lexical Ambiguity

- She is looking for a **match**
- The fisherman went to the **bank**

Syntactic Ambiguity

- The chicken is ready to eat
- Visiting relatives can be boring
- I saw the man with the binoculars

Referential Ambiguity

- The boy told his father the theft. **He** was very upset

Preprocessing in NLP

Tokenization

Today

we

will

understand

Tokenization

01

Break a complex sentence into words



02

Understand the importance of each of the words with respect to the sentence

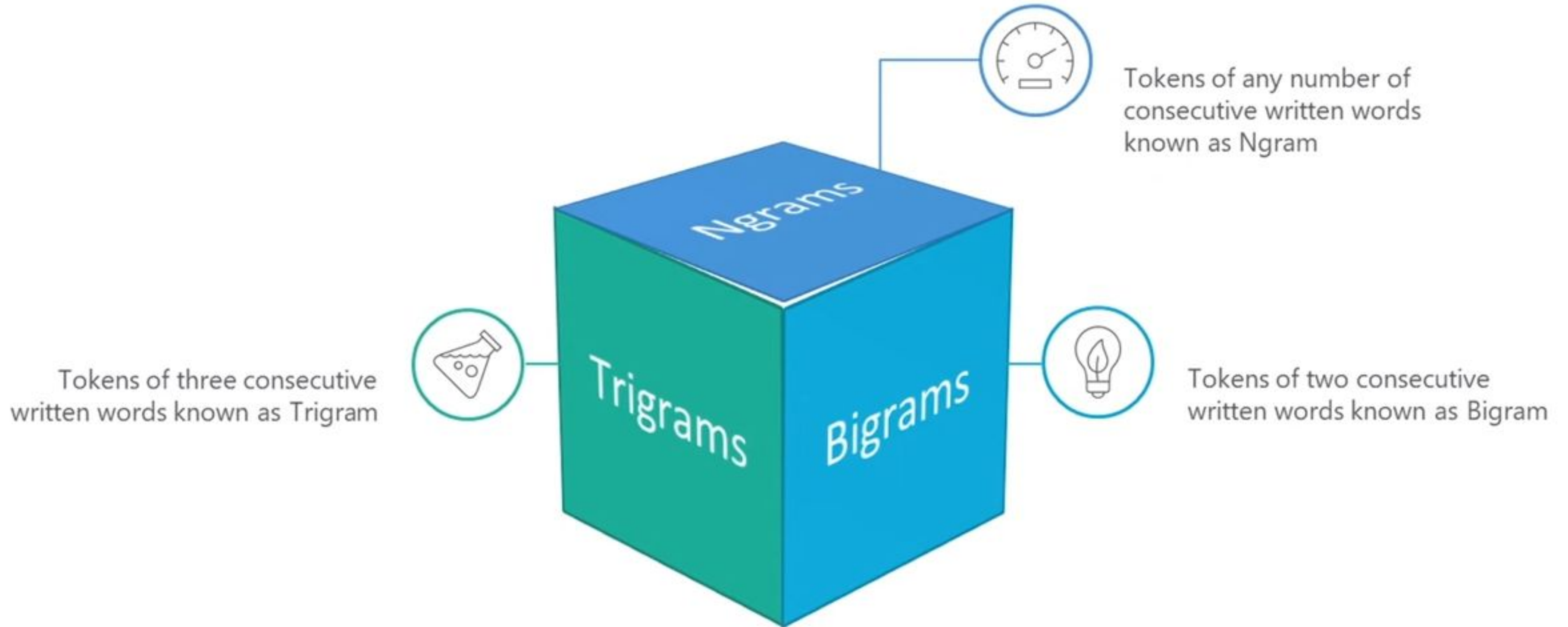


03

Produce a structural description on an input sentence



Tokenization



Stemming



Normalize words into its base form or root form

Affectation

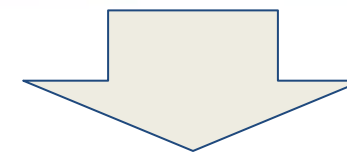
Affects

Affections

Affected

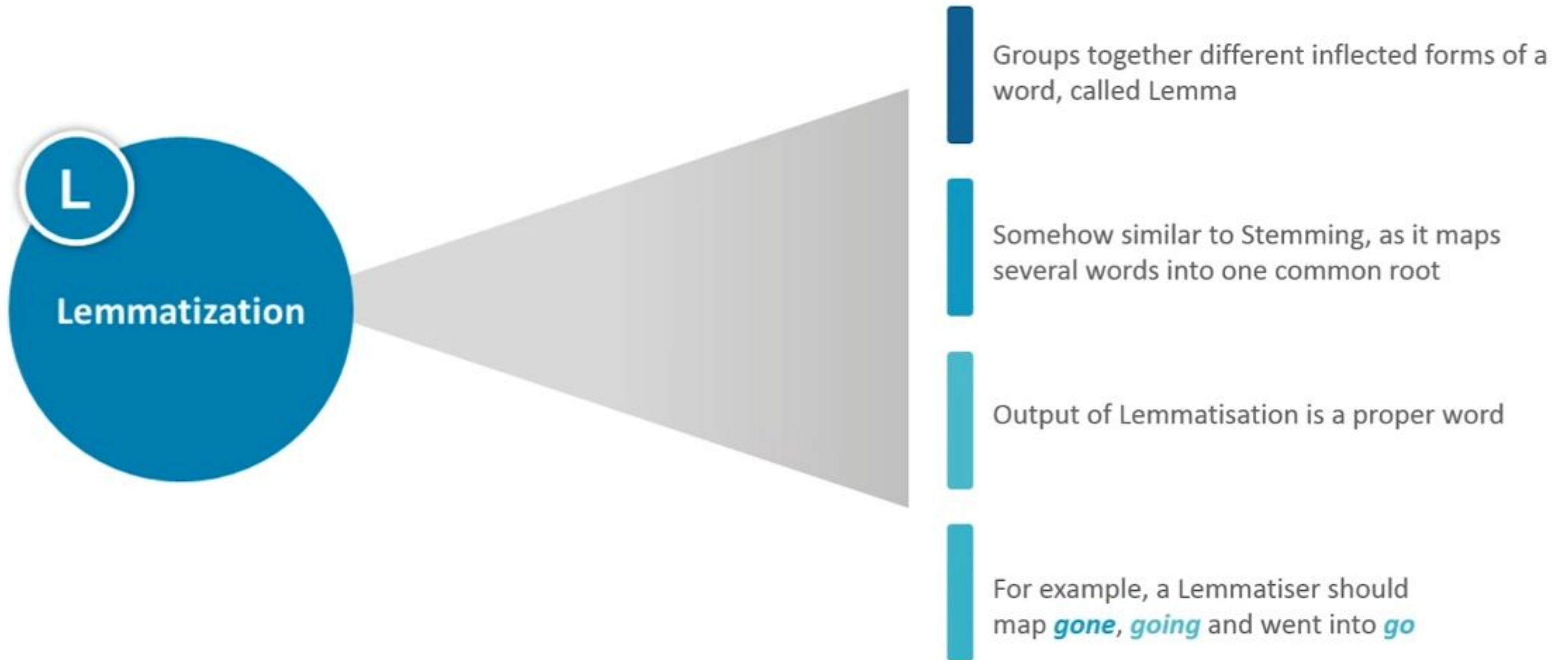
Affection

Affecting



Affect

Lemmatization



Stemming vs Lemmatization

Stemming	Lemmatization
Stemming is a process that stems or removes last few characters from a word, often leading to incorrect meanings and spelling.	Lemmatization considers the context and converts the word to its meaningful base form, which is called Lemma.
For instance, stemming the word ' Caring ' would return ' Car '.	For instance, lemmatizing the word ' Caring ' would return ' Care '.
Stemming is used in case of large dataset where performance is an issue.	Lemmatization is computationally expensive since it involves look-up tables and what not.

Stop Word



Are they Helpful or Not ?



Remove Punctuation

ENGLISH PUNCTUATION

,

comma

;

semicolon

-

en dash

—

em dash

-

hyphen

“ ”

quotation
marks

:

colon

'

apostrophe

!

exclamation
point

?

question
mark

...

ellipsis

()

parantheses

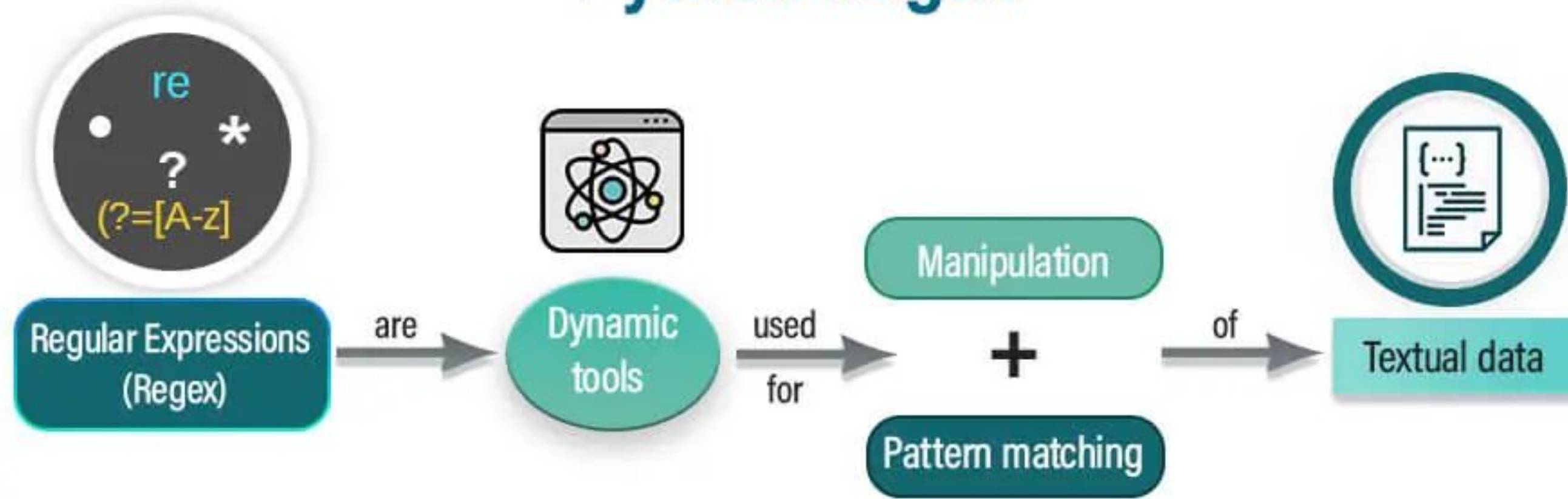
.

period



Regular Expressions (Regex)

Python Regex



Searching

Extracting

Replacing

Splitting

Validating

Text Vectorization - BoW

Bag of Words (BoW)

"I like oranges, do you like oranges?"



BoW text vector



apples
do
I
like
oranges
you

Example Sentiment Analysis

- Tweet 1: "I love this product!"
- Tweet 2: "This product is terrible."

BoW matrix:

	I	love	this	product	is	terrible
Tweet 1	1	1	1	1	0	0
Tweet 2	0	0	1	1	1	1

Example Spam Filtering

- Email 1: "You have won a million dollars! Click here to claim your prize."
- Email 2: "The company meeting will be held next Monday."

BoW matrix:

	you	have	won	a	million	dollars	click	here	to	claim	your	prize
Email 1	1	1	1	1	1	1	1	1	1	1	1	1
Email 2	0	0	0	0	0	0	0	0	0	0	0	0

Text Vectorization - TF-IDF

Term frequency - Inverse document frequency (TF-IDF)

$$w_{x,y} = \text{tf}_{x,y} \times \log \left(\frac{N}{\text{df}_x} \right)$$

TF-IDF

Term x within document y

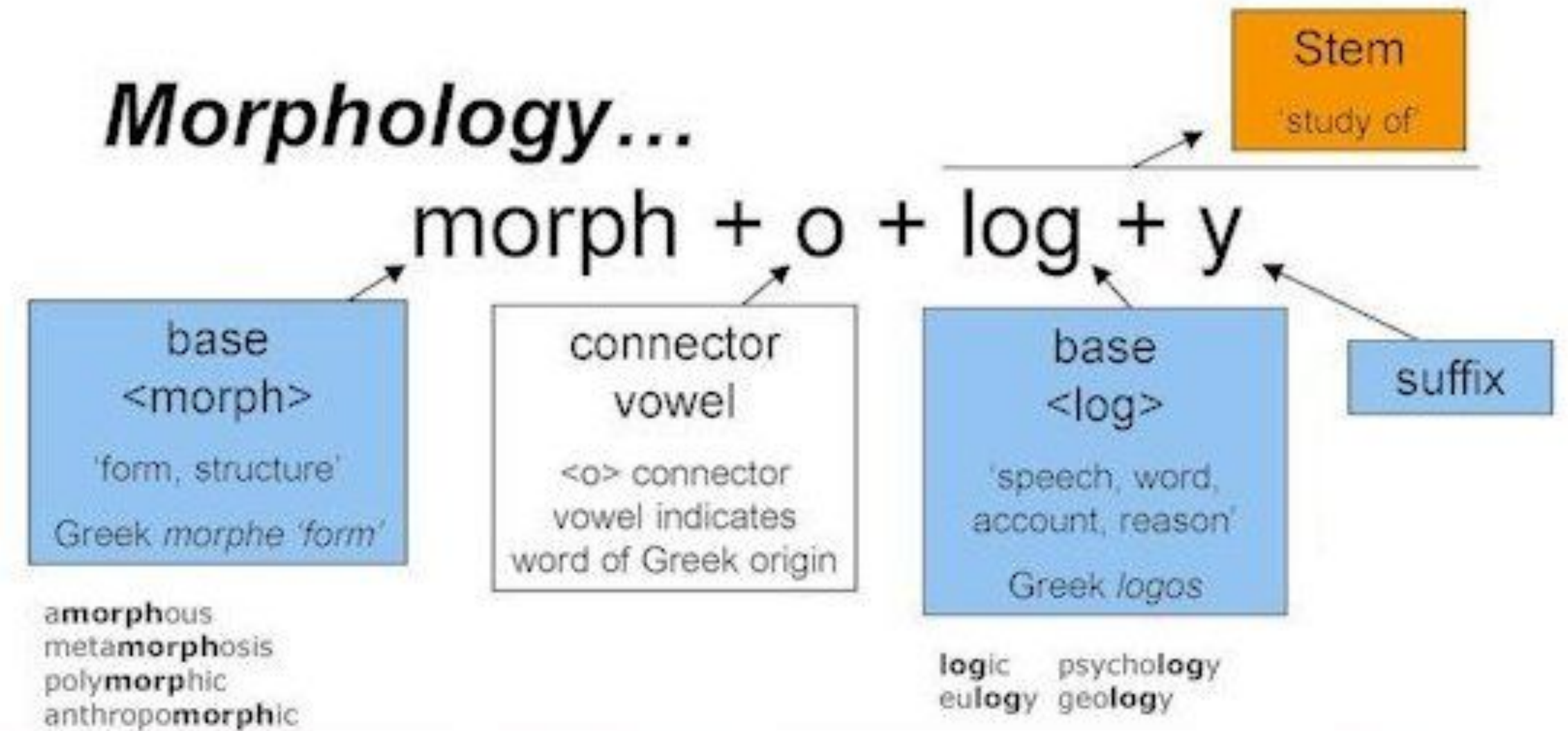
$\text{tf}_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

Morphological segmentation

Morphology...



government

--> govern - ment - s

Morphemes

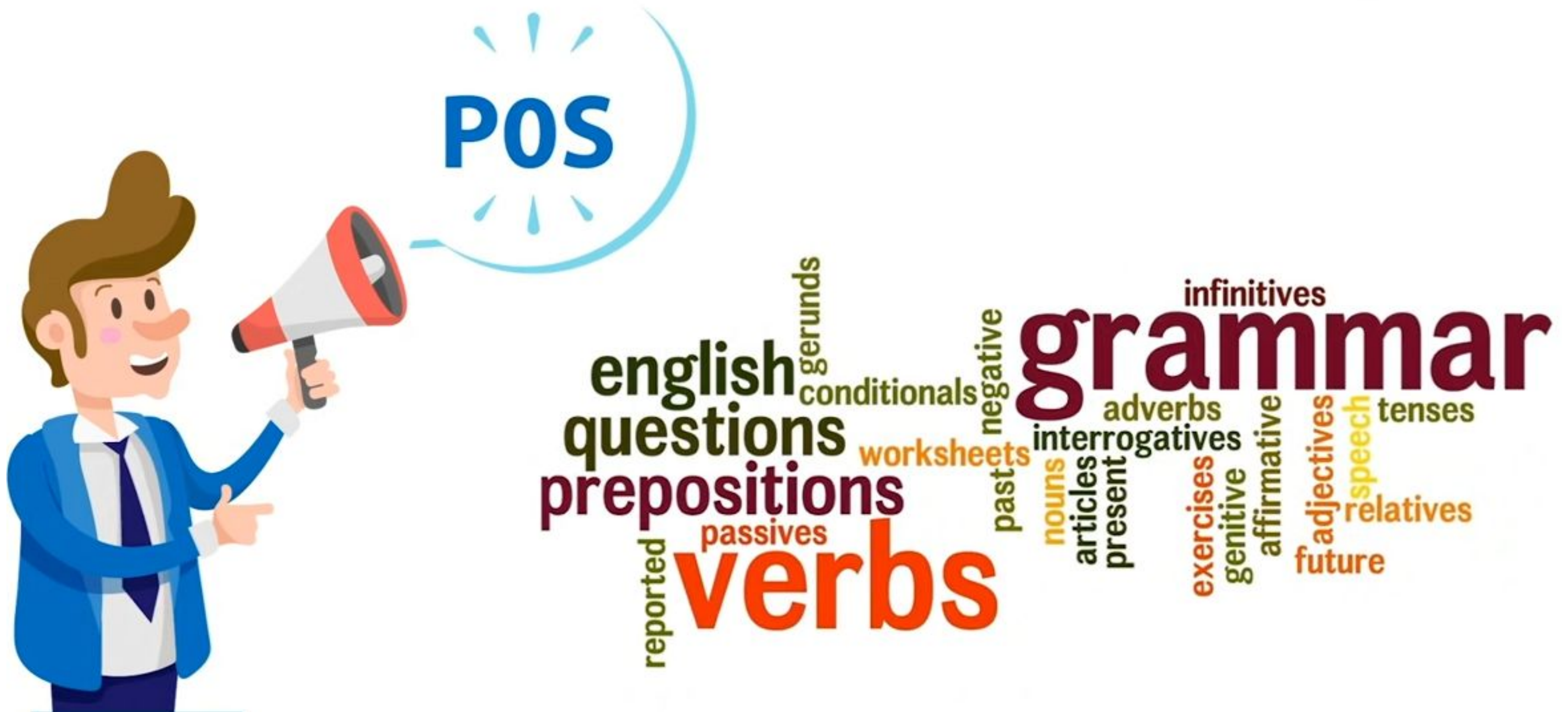
- smallest units that can carry meaning in a word
- bases, affixes (prefixes or suffixes), connector vowels
- "The letter or letters between plus signs in a word sum."
(working def. with my grade 4 students)

base

Via Latin *basis*
'base, pedestal', from Greek

Syntactic Analysis

POS - Part of Speech



POS - Part of Speech

Tag	Description
CC	Coordinating conjunction
CD	Cardinal number
DT	Determiner
EX	Existential there
FW	Foreign word
IN	Preposition or subordinating conjunction
JJ	Adjective
JJR	Adjective, comparative
JJS	Adjective, superlative
LS	List item marker
MD	Modal
NN	Noun, singular or mass
NNS	Noun, plural
NNP	Proper noun, singular
NNPS	Proper noun, plural
PDT	Predeterminer
POS	Possessive ending
PRP	Personal pronoun

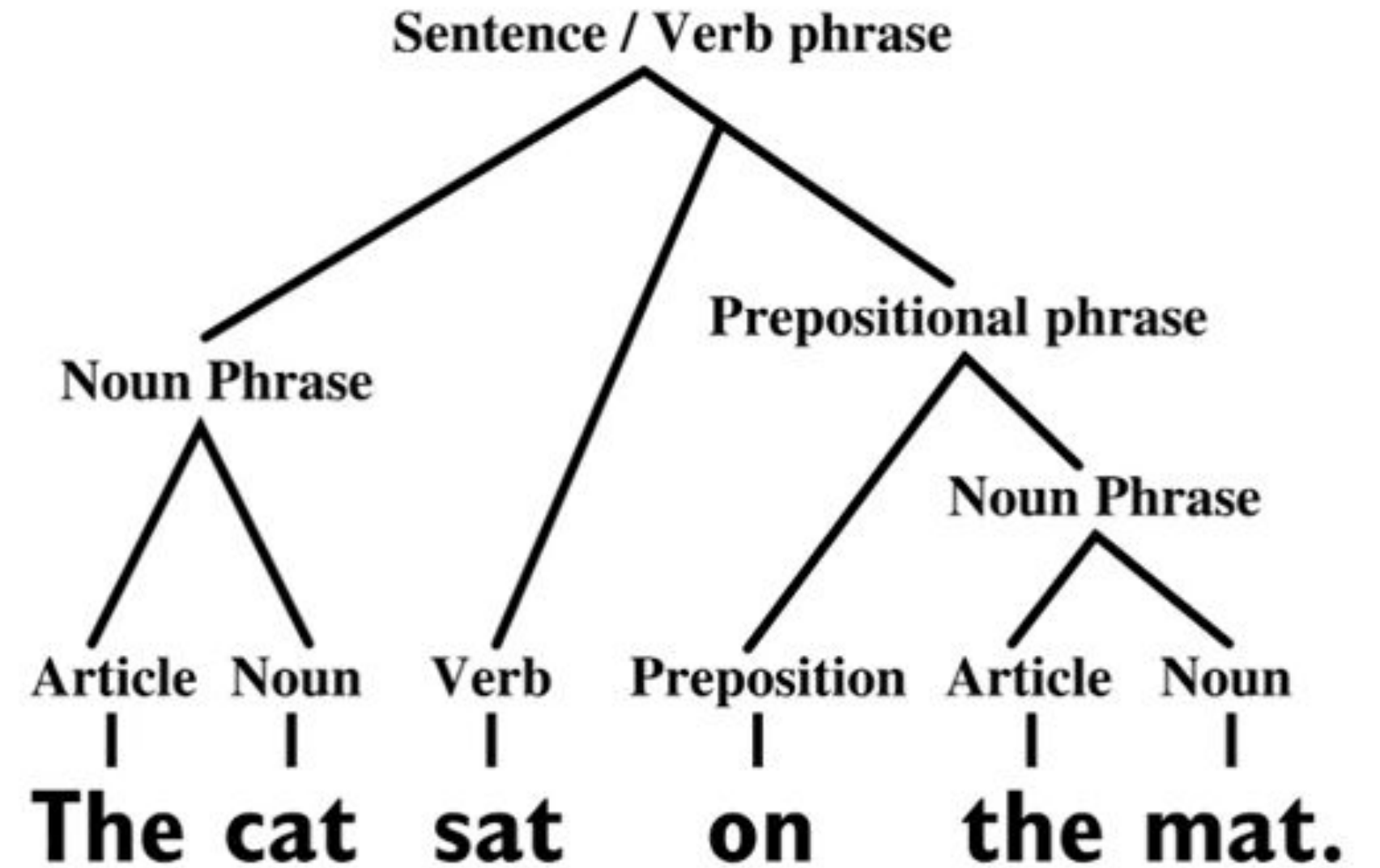
Tag	Description
PRP\$	Possessive pronoun
RB	Adverb
RBR	Adverb, comparative
RBR	Adverb, superlative
RP	Particle
SYM	Symbol
TO	to
UH	Interjection
VB	Verb, base form
VBD	Verb, past tense
VBG	Verb, gerund or present participle
VCN	Verb, past participle
VBP	Verb, non3rd person singular present
VBZ	Verb, 3rd person singular present
WDT	Whdeterminer
WP	Whpronoun
WP\$	Possessive whpronoun
WRB	Whadverb

Syntax Tree

Syntax Tree is a tree representation of *syntactic structure of sentences* or strings

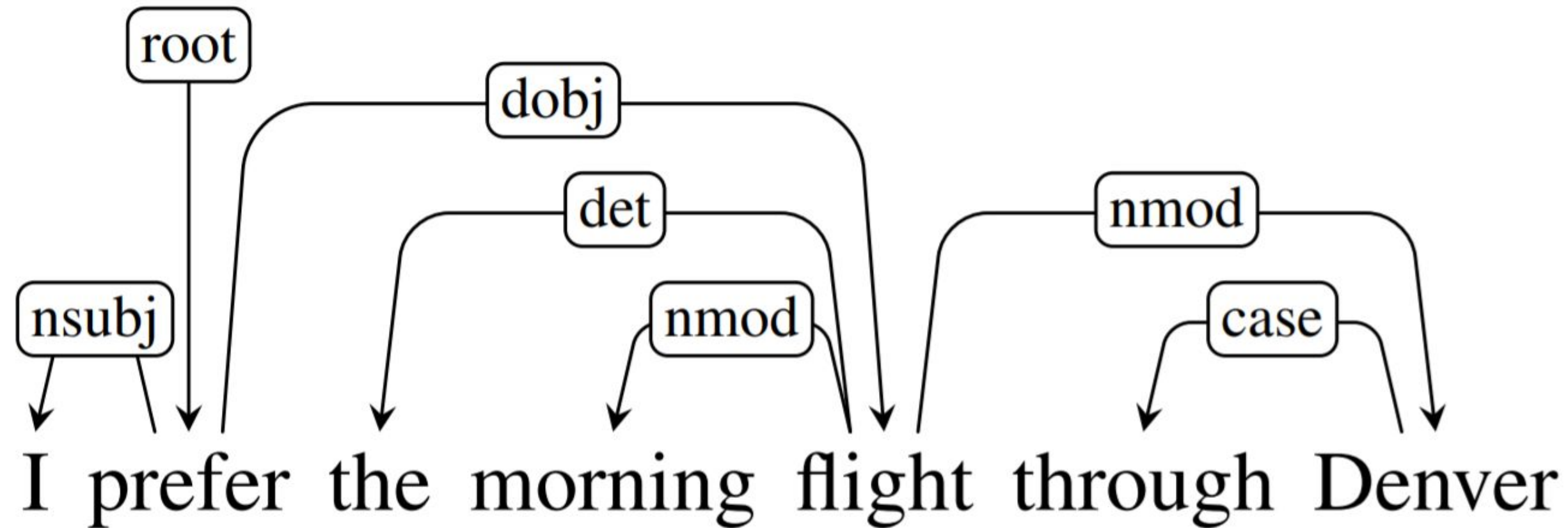


Basic constituent structure analysis of a sentence:



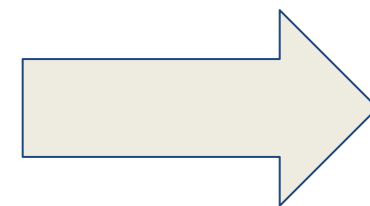
Dependency Parsing

Examining the dependencies between the phrases of a sentence



Dependency Tag Description

acl	clausal modifier of a noun (adnominal clause)
acl:relcl	relative clause modifier
advcl	adverbial clause modifier
advmod	adverbial modifier
advmod:emph	emphasizing phrase, intensifier
advmod:lmod	locative adverbial modifier
amod	adjectival modifier
appos	appositional modifier
aux	auxiliary
aux:move	passive auxiliary
case	case-marking
cc	coordinating conjunction
cc:preconj	preconjunct
ccomp	clausal complement
clf	classifier
compound	compound



37 common syntactic **relationships**

Semantic Analysis

Semantic Analysis

Understanding the meaning, interpretation, and relationships between words, phrases, and sentences in a **given context**

Where does Semantic Analysis Work?



Natural Language Processing (NLP)



Search Engines



Information Retrieval



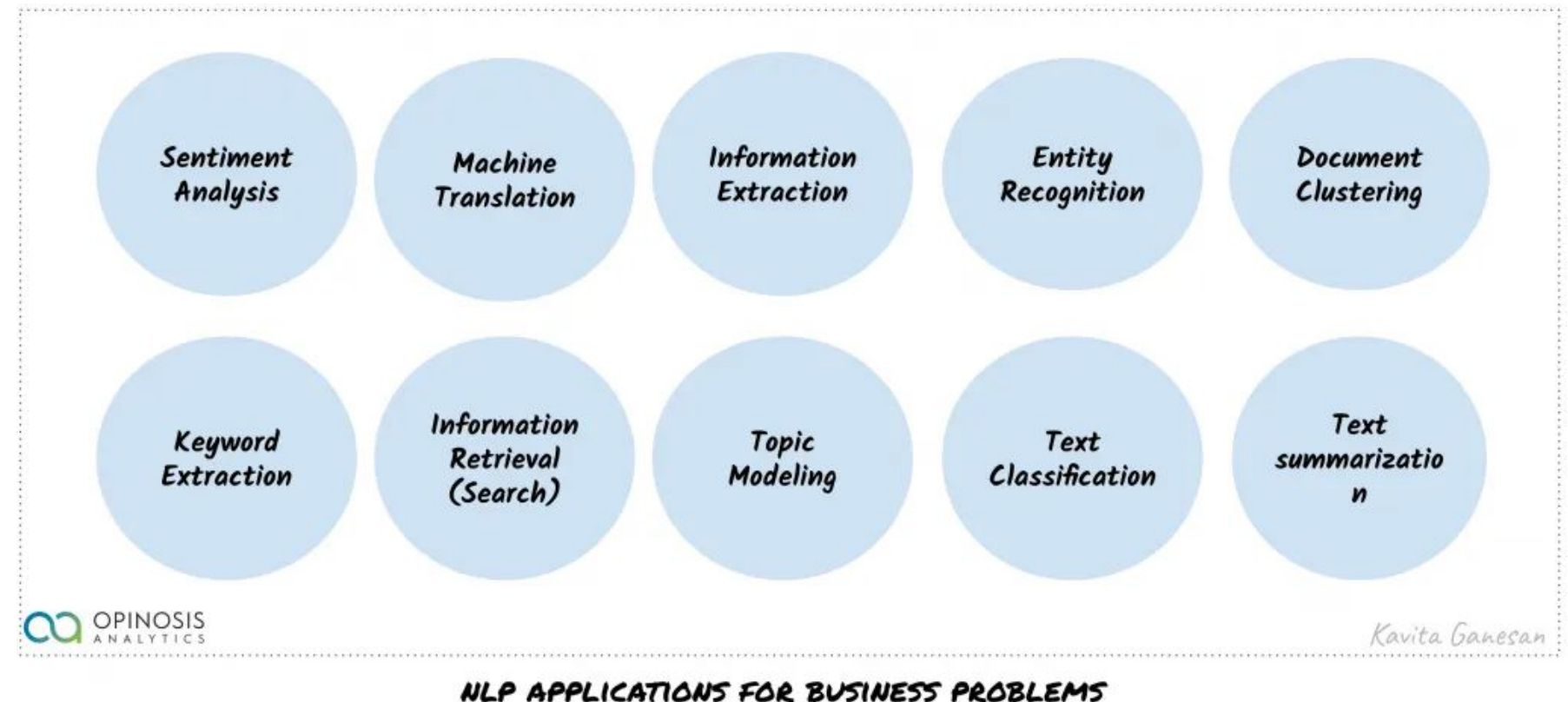
Chatbots and Virtual Assistants



Machine Learning and AI



Customer Service and Support



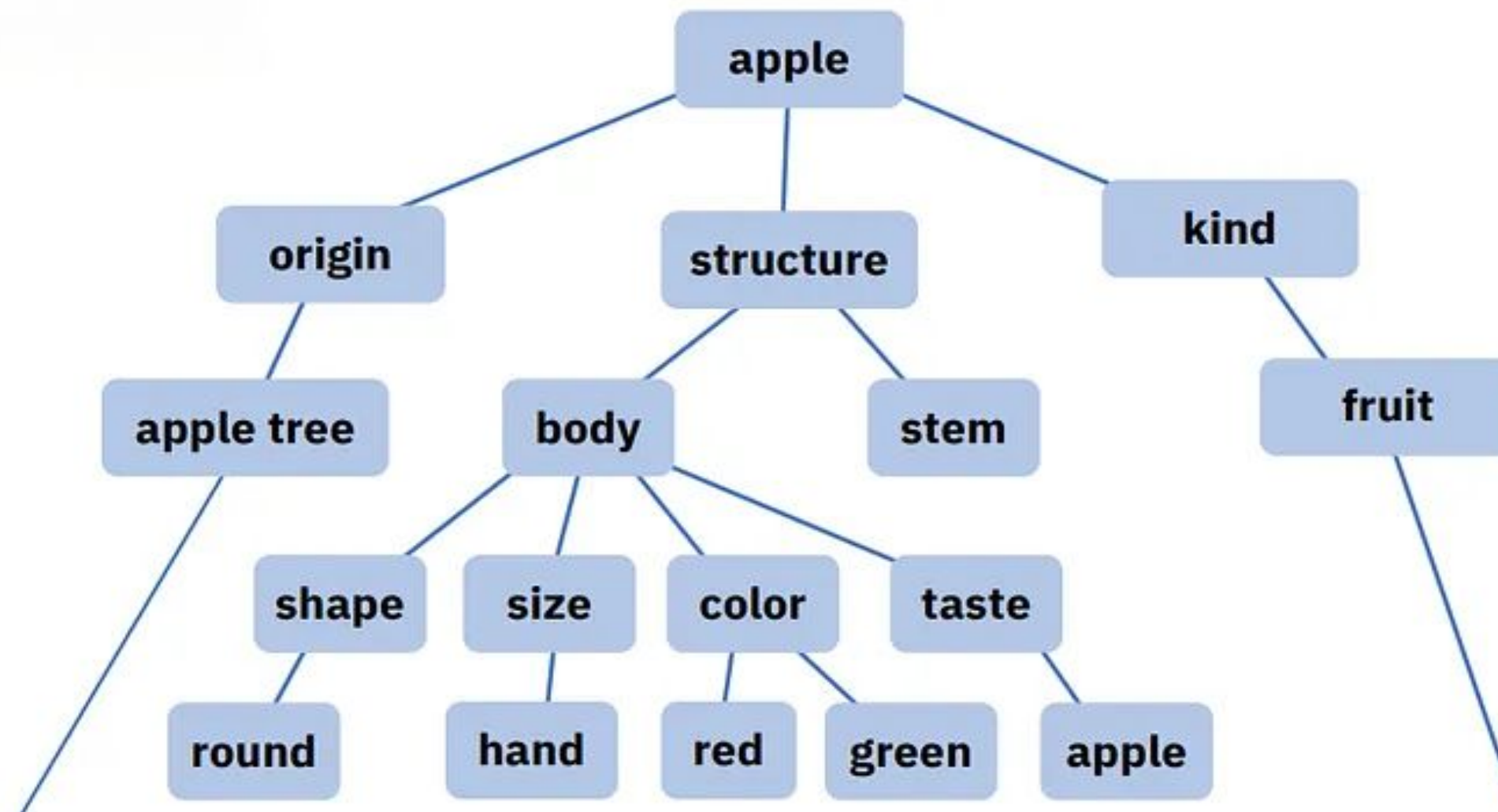
History of Natural Language Processing



- Symbolic NLP
- Statistical NLP
- Neural NLP

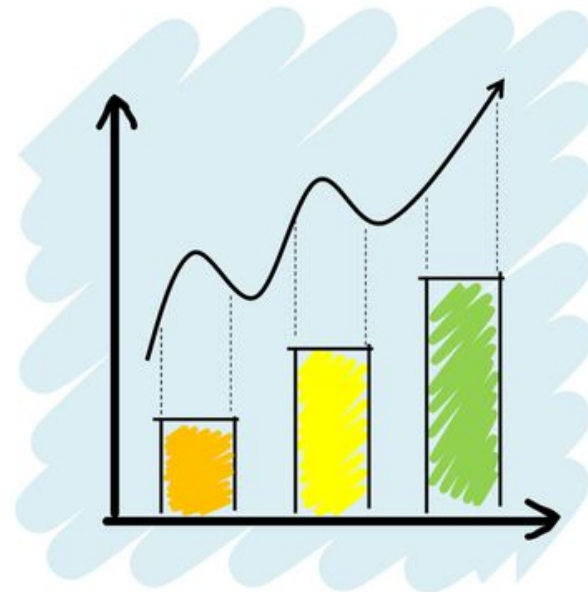
Symbolic NLP

- Given a collections of rule
- The computer emulates natural language understanding (or other NLP tasks) by applying those rules to the data it confronts.



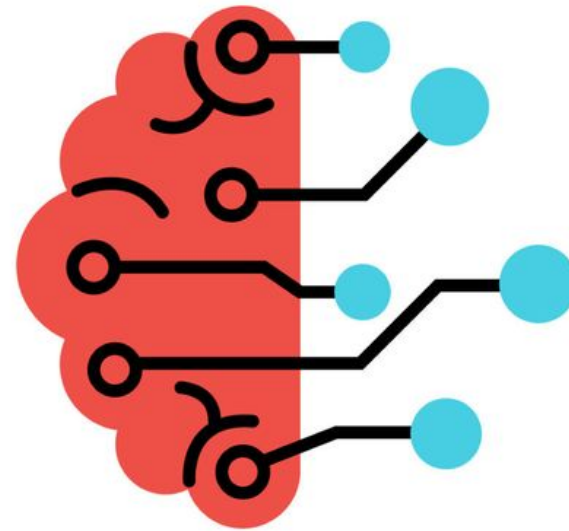
Statistical NLP

- Use the large amount of data
- Learn the probability on that corpus (for example frequency of word “Thank” go along with “you”)
- N-gram is a statistical NLP method



Neural NLP

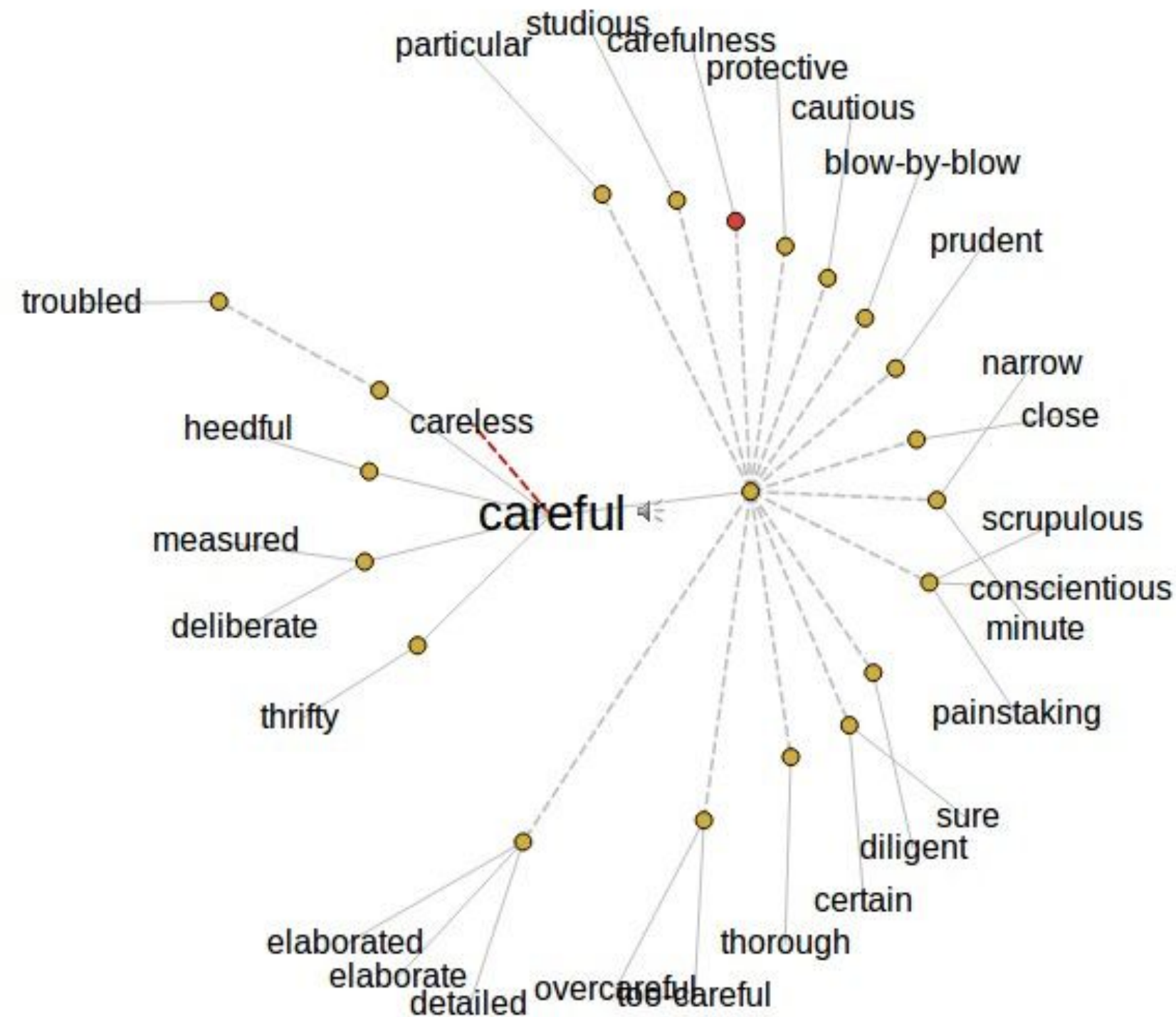
- Appli multi-layer neural network into NLP
- Use large amount of data and the neural network capability to learn the complex pattern in text automatically.



NEURAL NETWORK

Word Embedding

How to present a word



- We expect the word represent that satisfy these properties:
 - Computer can use it
 - Can compute similarity between many words

What we expect



Word Embedding

One-hot-vector

Such symbol can be represented as an one-hot vector:

```
motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
```

Vector dimension = number of words in vocab

Disadvantages

- These vector are orthogonal
- There is no natural notion of similarity for one-hot vector, ex: men vs man
- Sparse vector, most dimensions of a one-hot vector hold a value of 0.
- High memory usage

Word Embedding

Distributional semantics

- The words with similar meanings tend to occur in similar contexts
- The word's meaning is given by the words that frequently appear close-by.
- When a word(w) appears in a text, its context is a set of word that appear nearby (fixed window size).
- Use the many context of " w " to build up the representation of " w "

Example

*...government debt problems turning into banking crises as happened in 2009...
...saying that Europe needs unified banking regulation to replace the hodgepodge...
...India has just given its banking system a shot in the arm...*

← context →

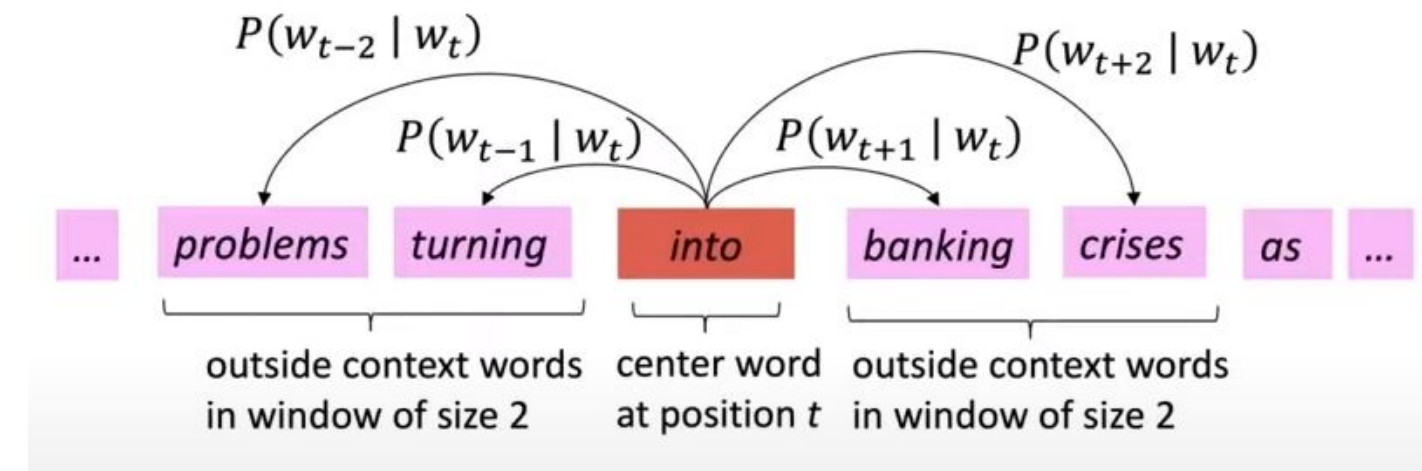
Dense vector of banking

$$\text{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Word Embedding

Word2vec

- We have large corpus of text
- Every word in a fixed vocabulary is represented by a vector
- Go through each position t in the text, which has center word c and context word (“outside”) o .
- Use the similarity of the word vectors for c and o to calculate the probability of o given c (or vice versa).
- Keep adjusting the word vectors to maximize this probability.

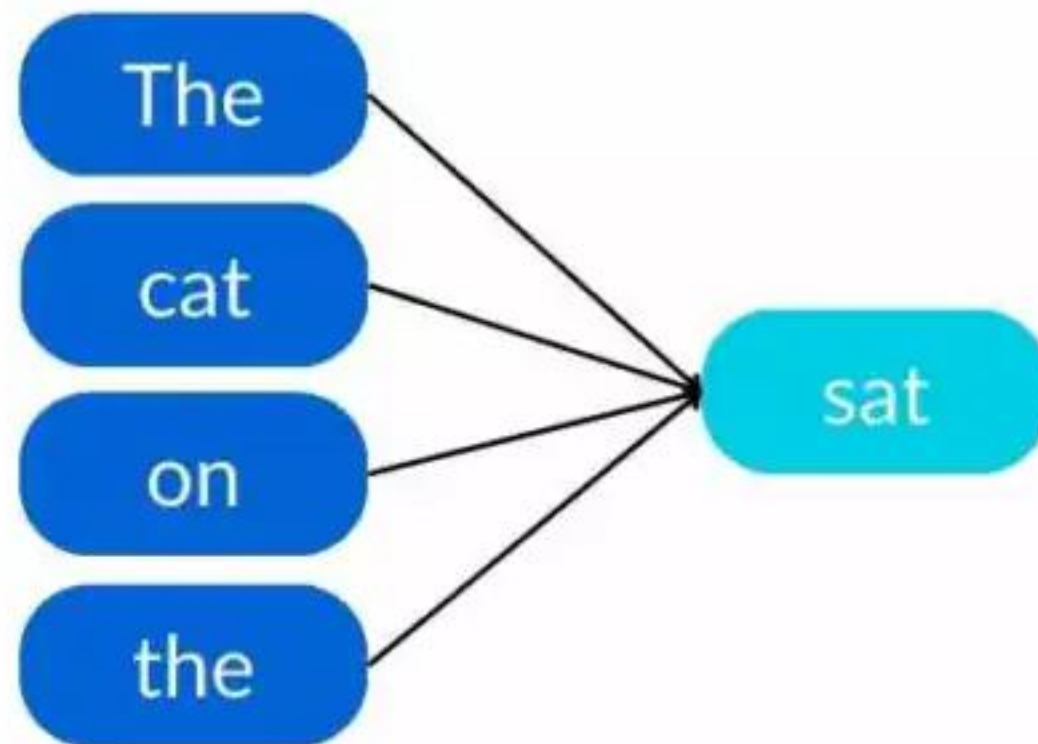


Word Embedding

Example Sentence: The cat sat on the mat.

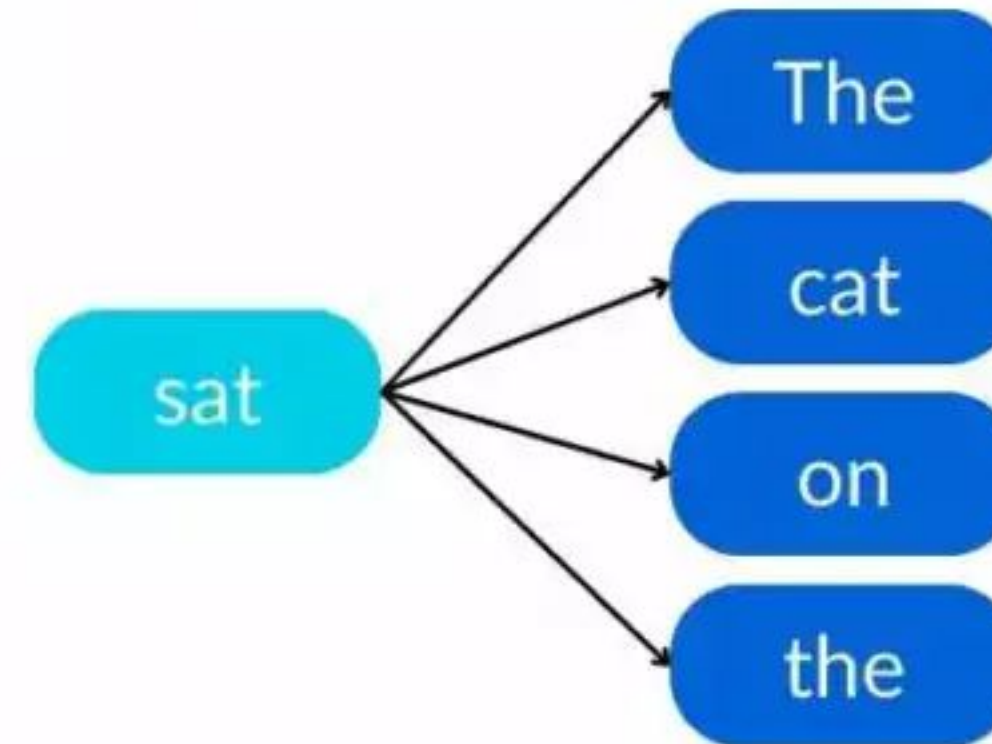
Continuous Bag-of-Words (CBOW)

Goal: Given context words,
predict the target word.



Skip-gram Model

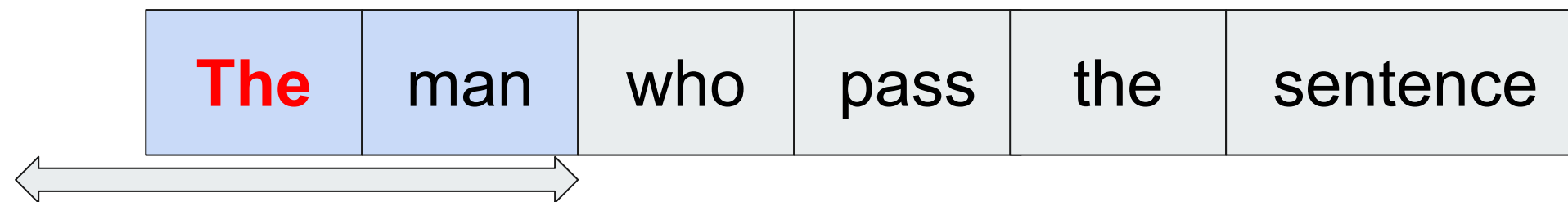
Goal: Given a word,
predict the surrounding context words.



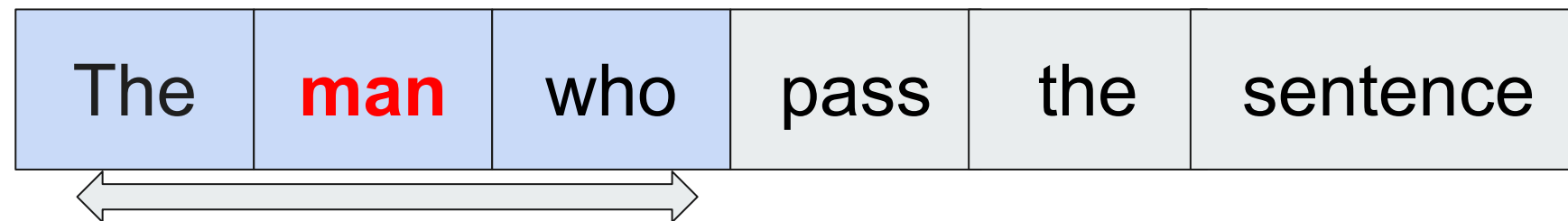
Word Embedding

Data

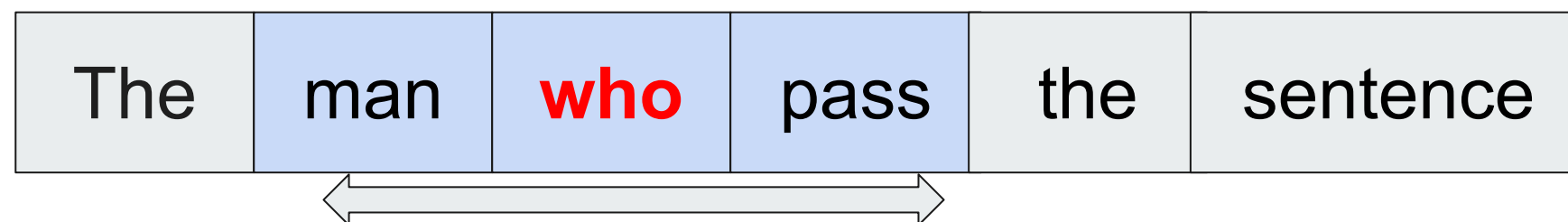
window size = 3



→ (the, man)



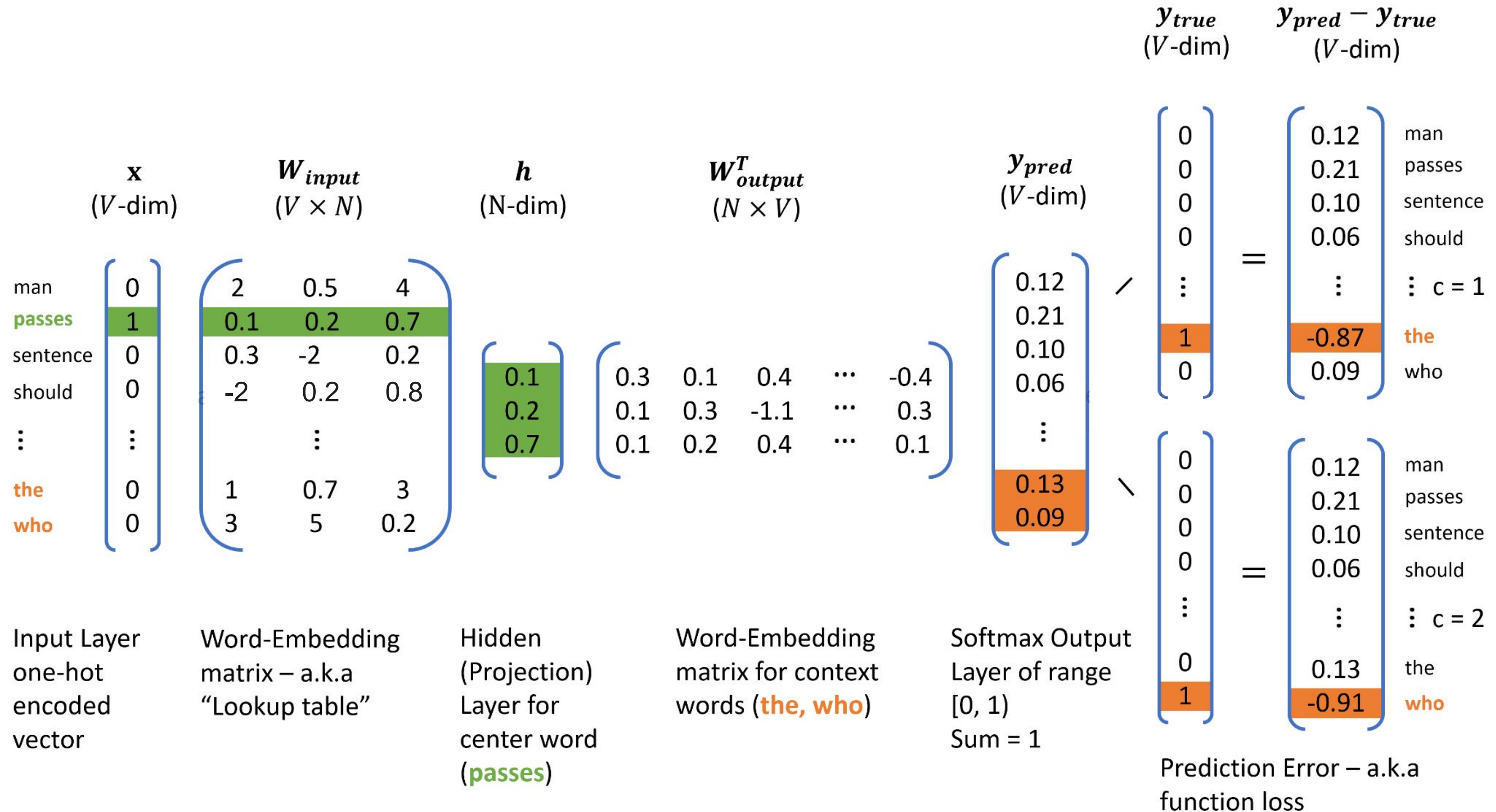
→ (the, man), (man, who)



→ (man, who), (who, pass)

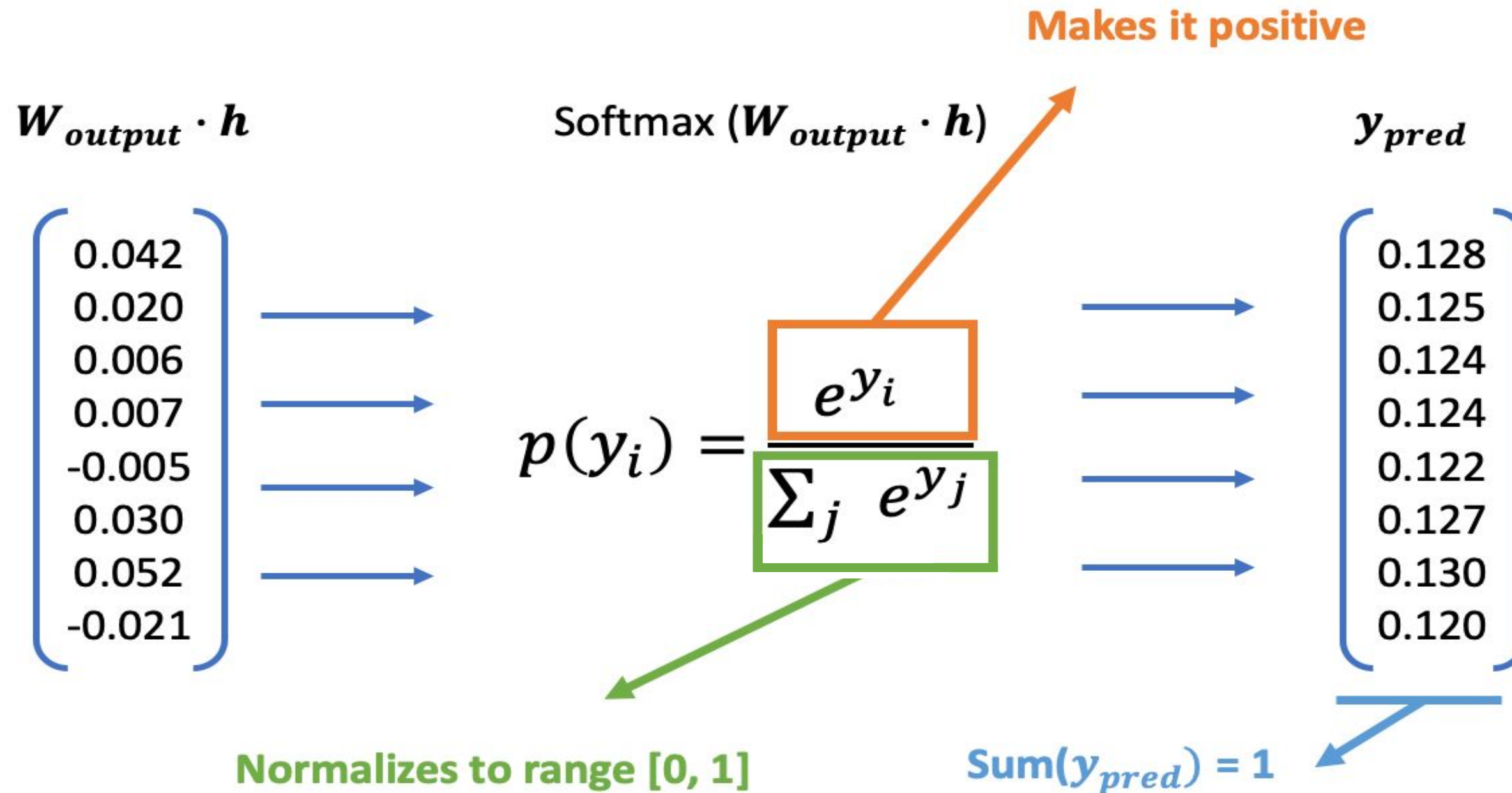
...

Word Embedding



Word Embedding

Softmax function transformation



Word Embedding

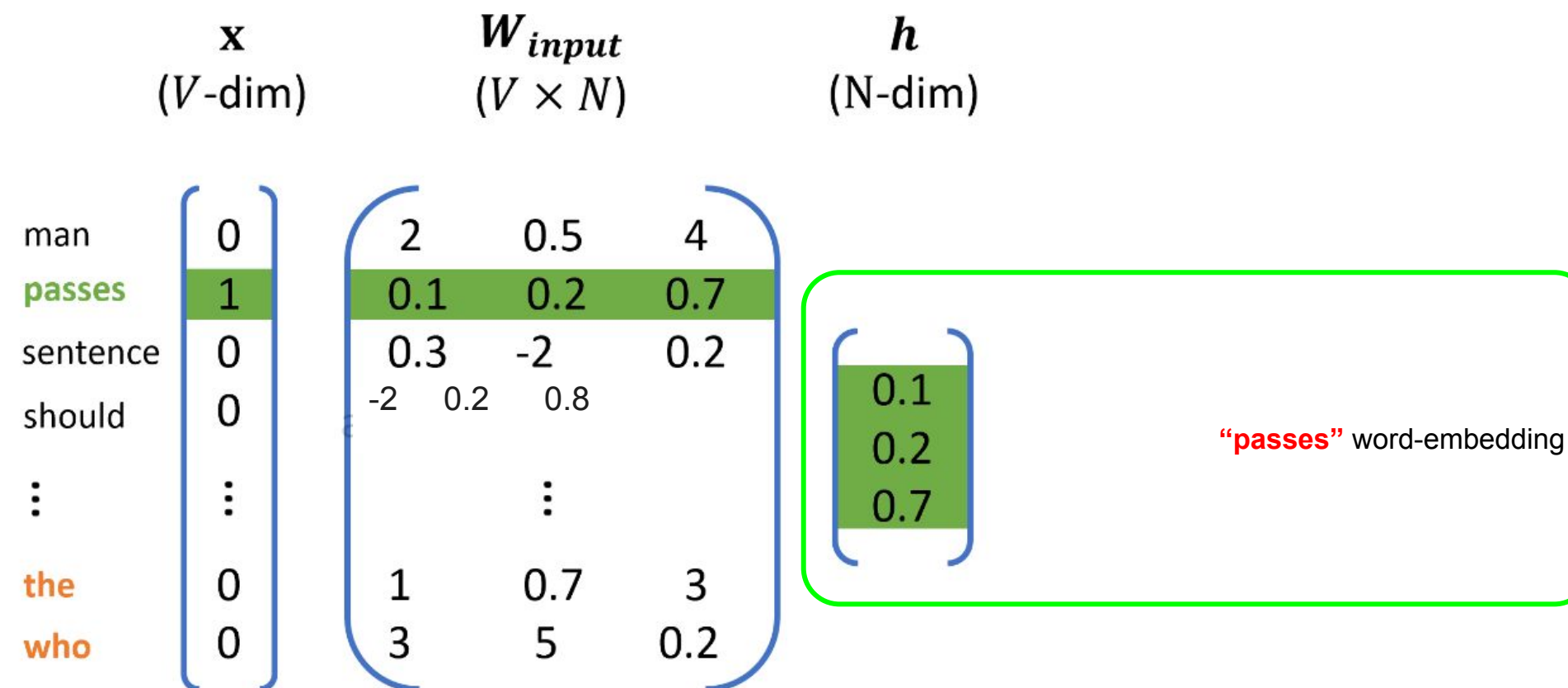
Prediction error window

$$\begin{array}{c}
 \mathbf{y}_{pred} \\
 (V\text{-dim})
 \end{array}
 \begin{array}{c}
 / \\
 \backslash
 \end{array}
 \begin{array}{c}
 \mathbf{y}_{true} \\
 (V\text{-dim})
 \end{array}
 =
 \begin{array}{c}
 \mathbf{y}_{pred} - \mathbf{y}_{true} \\
 (V\text{-dim})
 \end{array}
 \begin{array}{c}
 \text{man} \\
 \text{passes} \\
 \text{sentence} \\
 \text{should} \\
 \vdots \\
 \text{c} = 1 \\
 \text{the} \\
 \text{who}
 \end{array}$$

$$\begin{array}{c}
 \begin{array}{c}
 0.12 \\
 0.21 \\
 0.10 \\
 0.06 \\
 \vdots \\
 0.13 \\
 0.09
 \end{array}
 \begin{array}{c}
 0 \\
 0 \\
 0 \\
 0 \\
 \vdots \\
 0 \\
 1
 \end{array}
 =
 \begin{array}{c}
 0.12 \\
 0.21 \\
 0.10 \\
 0.06 \\
 \vdots \\
 -0.87 \\
 0.09
 \end{array}
 \begin{array}{c}
 \text{man} \\
 \text{passes} \\
 \text{sentence} \\
 \text{should} \\
 \vdots \\
 \text{c} = 1 \\
 \text{the} \\
 \text{who}
 \end{array}
 \end{array}$$

Word Embedding

Word-embedding matrix



Input Layer
one-hot
encoded
vector

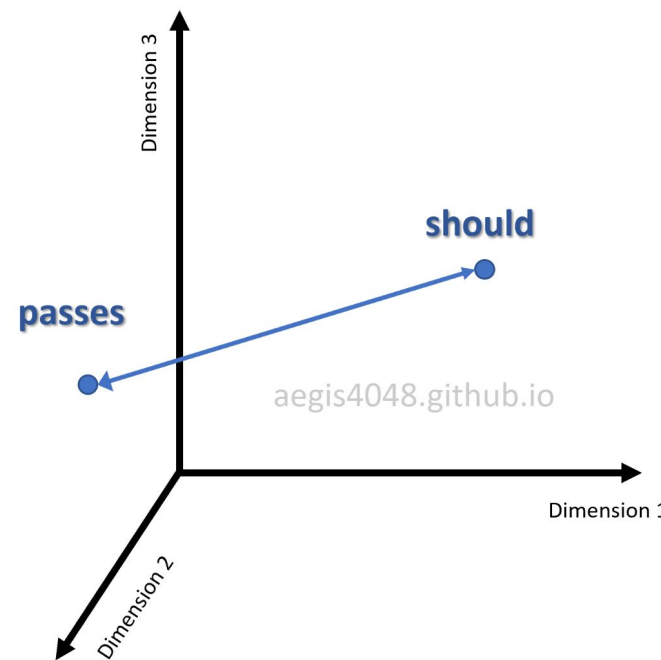
Word-Embedding
matrix – a.k.a
“Lookup table”

Hidden
(Projection)
Layer for
center word
(**passes**)

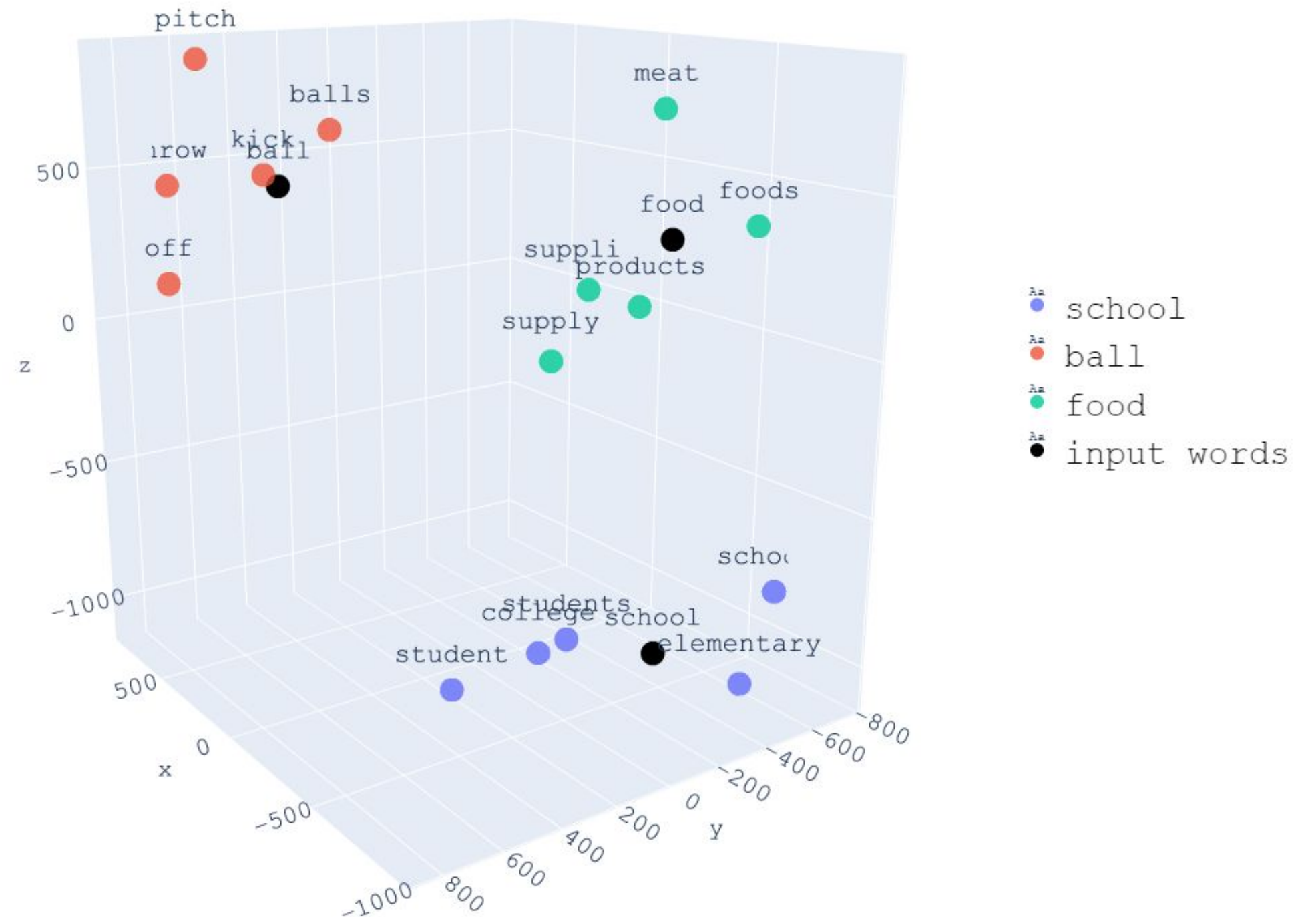
Word Embedding

W_{input}
($V \times N$)

man	2	0.5	4
passes	0.1	0.2	0.7
sentence	0.3	-2	0.2
should	-2	0.2	0.8
⋮	aegis4048.github.io		
the	1	0.7	3
who	3	5	0.2



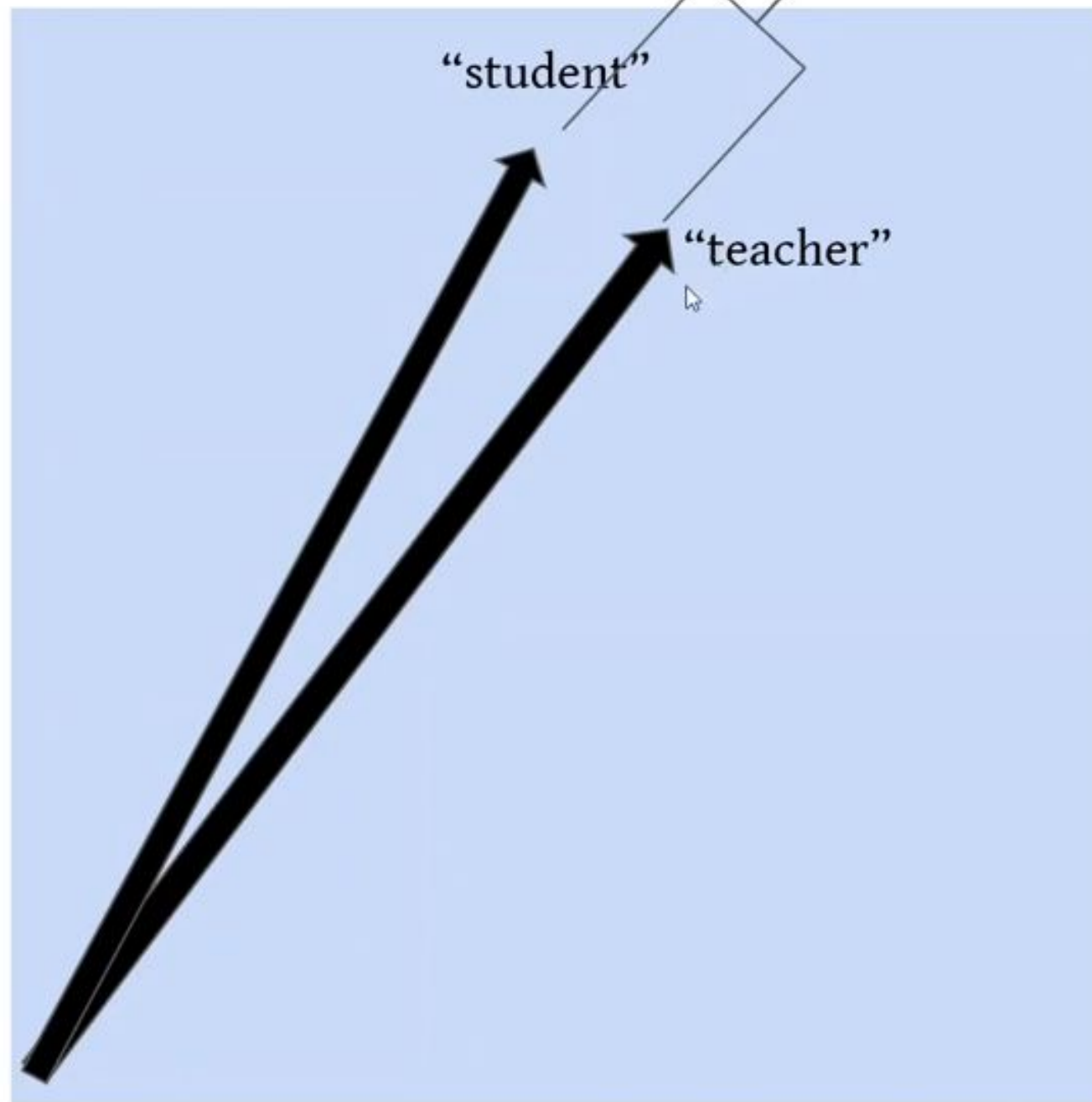
3D visualization of word vectors in embedding matrix



Visualization of Word2vec

Word Embedding

Vector visualization



One can determine the relevance of two words by computing the euclidean distance between their respective vectors:

$$d(\text{firstVec}, \text{secondVec}) = \sqrt{\sum_{i=1}^{\text{number of dimentions}} (\text{firstVec}_i - \text{secondVec}_i)^2}$$

Or cosine distance:

$$d(\text{firstVec}, \text{secondVec}) = 1 - \frac{\text{dot}(\text{firstVec}, \text{secondVec})}{\|\text{firstVec}\| \cdot \|\text{secondVec}\|}$$

Language Modeling

- Language modeling is the task of predicting what word comes next.

the students opened their _____

- Formally: given a sequence of words $x(1)$, $x(2), \dots, x(t)$, compute the probability:

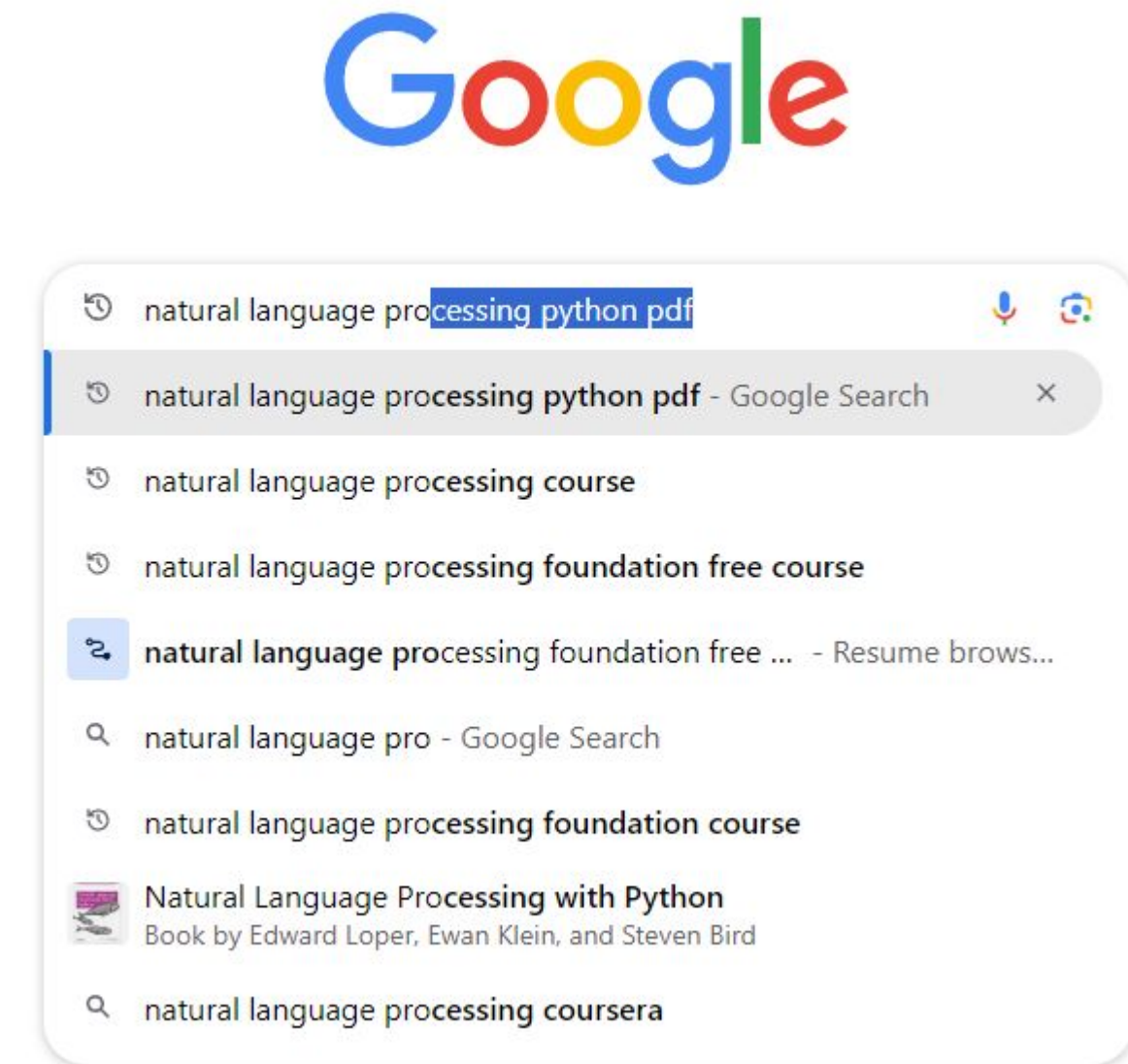
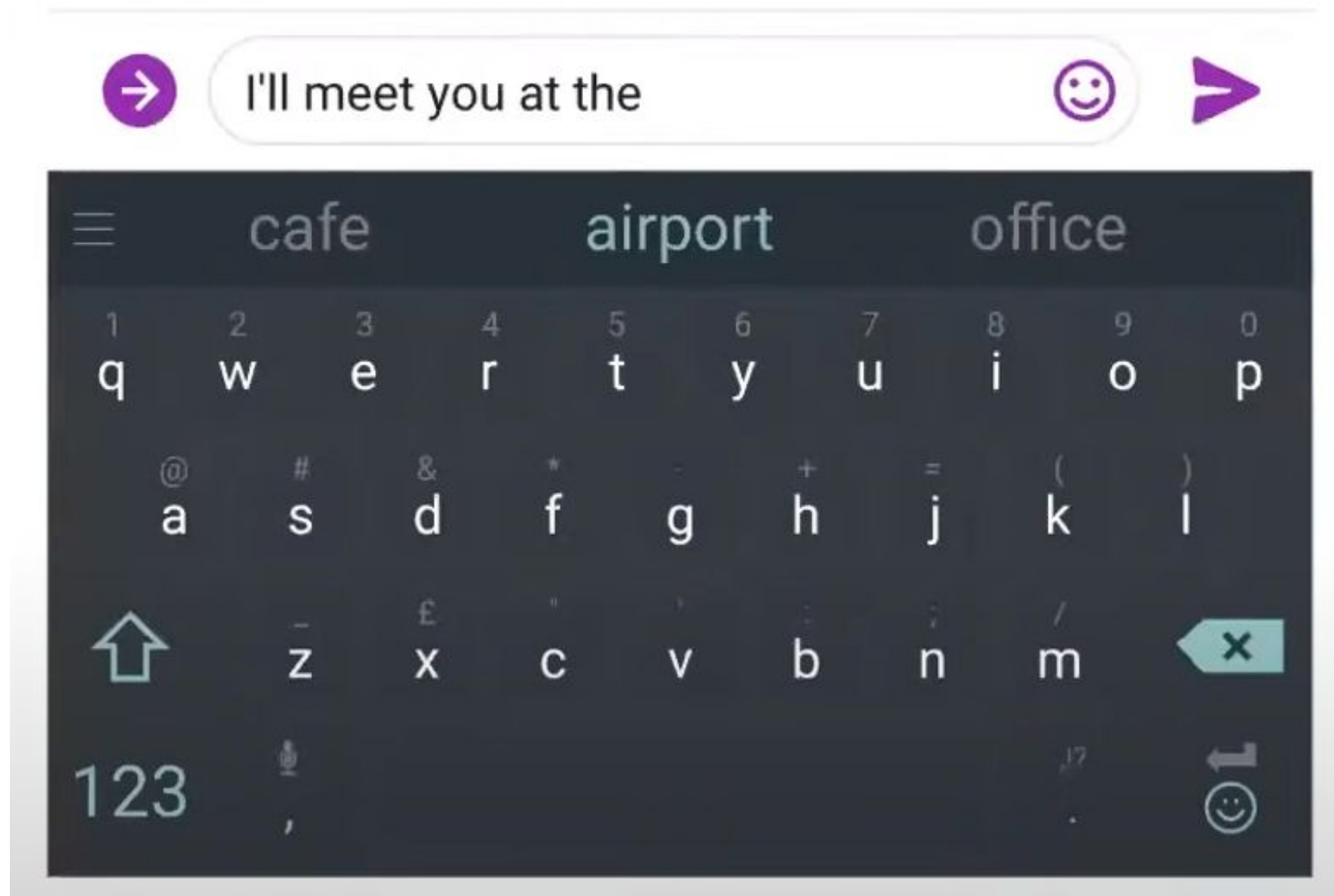
$$P(x^{t+1} | x^t, \dots, x^1)$$

- Where $x(t+1)$ can be any word of in the vocabulary.
- A system that does this is called a Language Model.



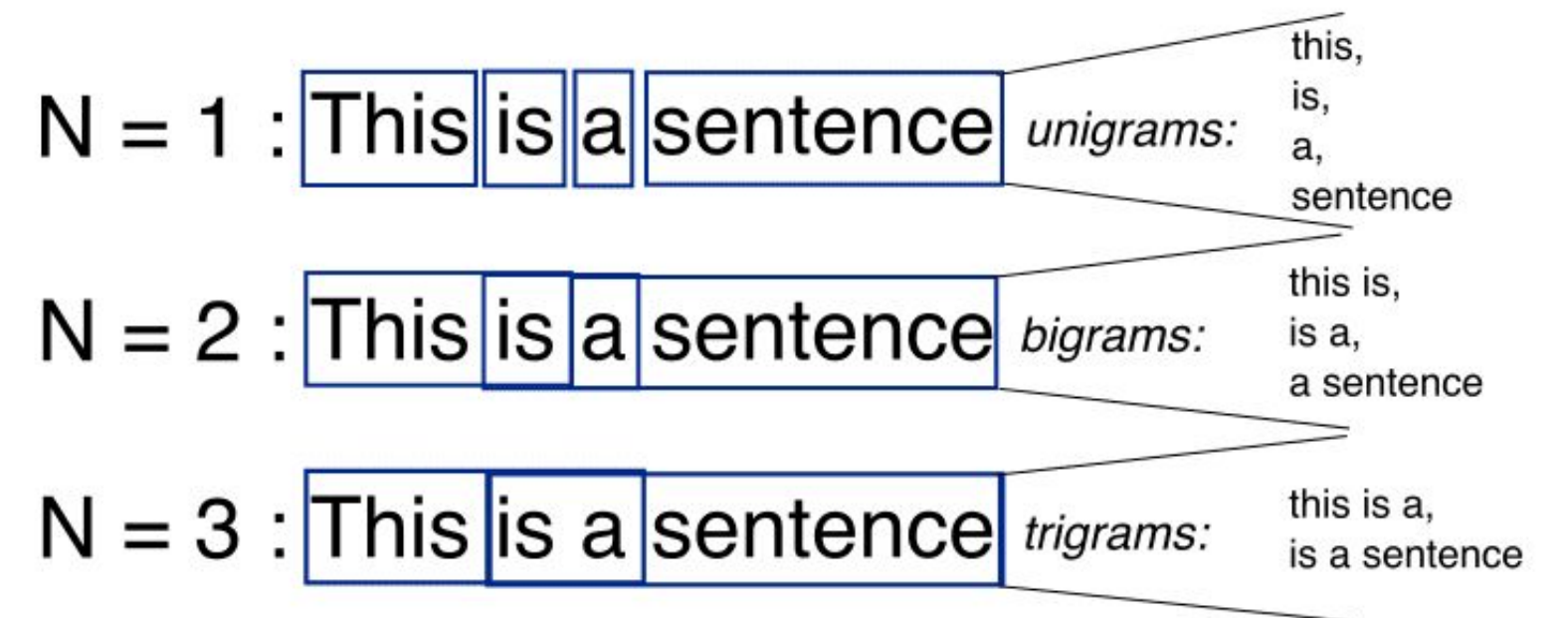
Language Modeling

We use language modeling everyday



N-gram language modeling

- Pre-Deep Learning: learn a n-gram Language Model!
- Definition: A n-gram is a chunk of n consecutive words.
 - unigram: “This”, “is”, “a”, “sentence”.
 - bi-gram: “This is”, “is a”, “a sentence”.
 - tri-gram: “This is a”, “is a sentence”.
- Idea: Collected statistics use to predict next word.



N-gram language modeling

Statistical aspect

- First we make Markov assumption: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(x^{t+1} | x^t, \dots, x^{t-n+2}) = \frac{P(x^{t+1}, x^t, \dots, x^{t-n+2})}{P(x^t, \dots, x^{t-n+2})}$$

- How to get these probabilities?
- By counting them in some large corpus of text

$$\approx \frac{\text{count}(x^{t+1}, x^t, \dots, x^{t-n+2})}{\text{count}(x^t, \dots, x^{t-n+2})}$$

This is the house that Jack built.

This is the malt

That lay in the house that Jack built.

This is the rat,

That ate the malt

That lay in the house that Jack built.

This is the cat,

That killed the rat,

That ate the malt

That lay in the house that Jack built.

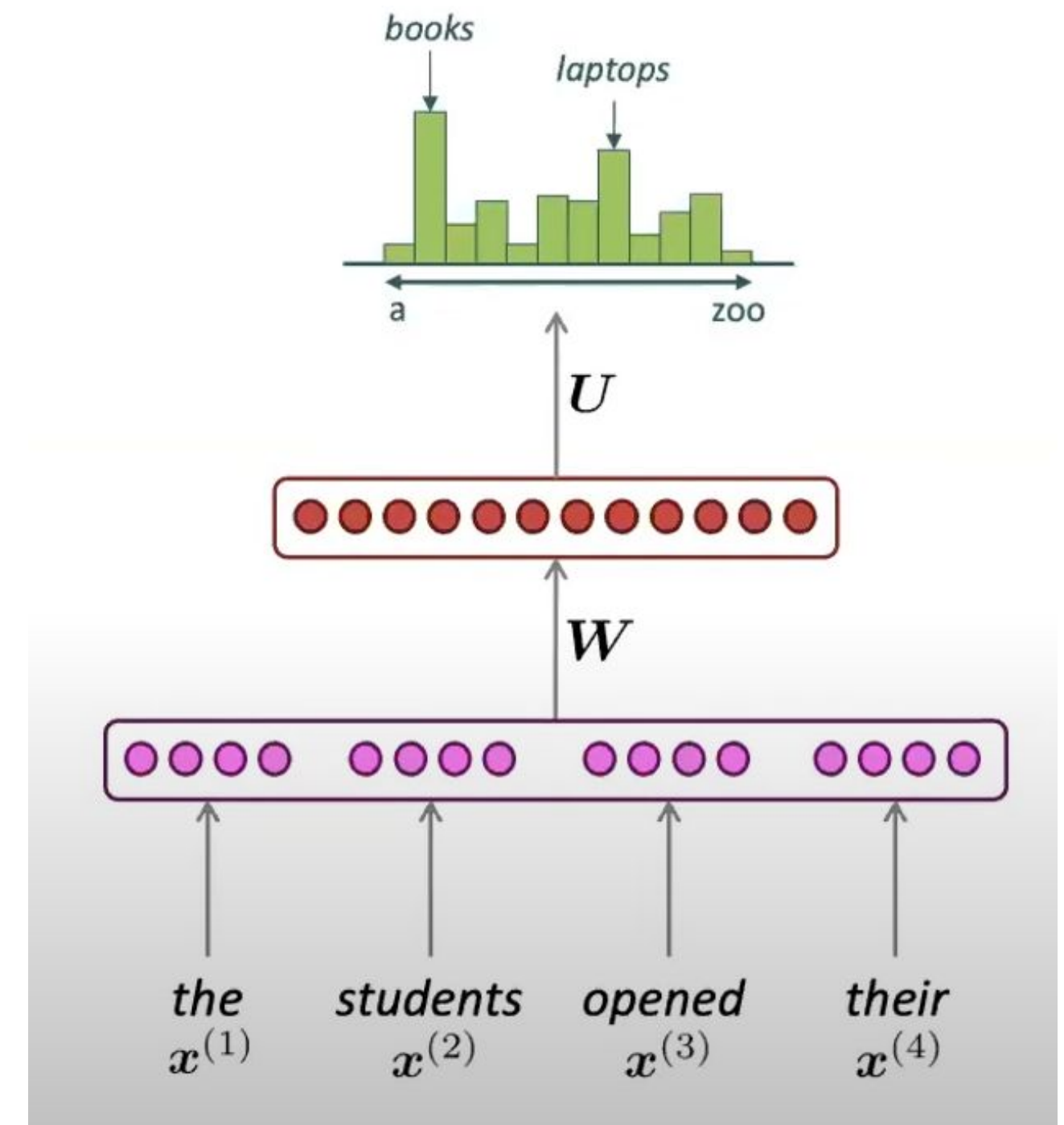
4-grams

$$p(\text{house} | \text{this is the}) = \frac{c(\text{this is the house})}{c(\text{this is the ...})} = \frac{1}{4}$$

N-gram language modeling

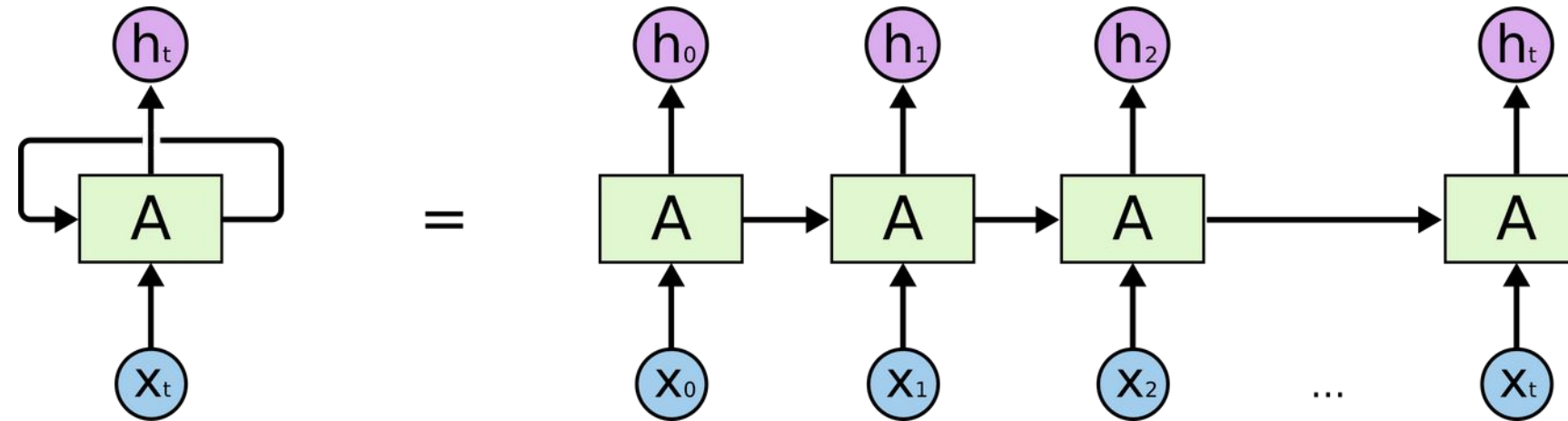
Fixed-window neural language model

- Improvement over n-gram
 - no sparsity problem
 - don't need to store all n-gram database
- Remaining problems:
 - Fixed-window is so small
 - Enlarging window enlarge W
 - Window can never large enough
 - No correlation between x
 - Order not matter



Recurrent Neural Network

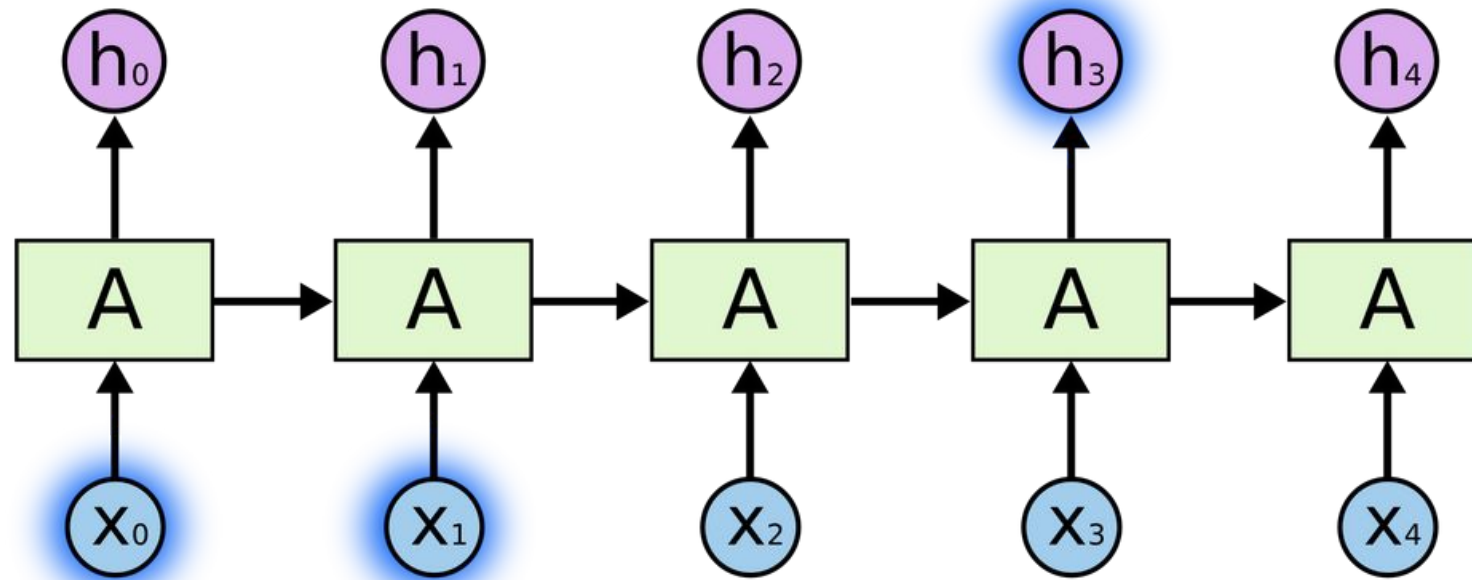
Architecture



- Improvement over fixed window NN
 - correlation between x
 - preserve order of information
 - infinity window size
- Remain problem
 - Forget Information

Recurrent Neural Network

Forget Information

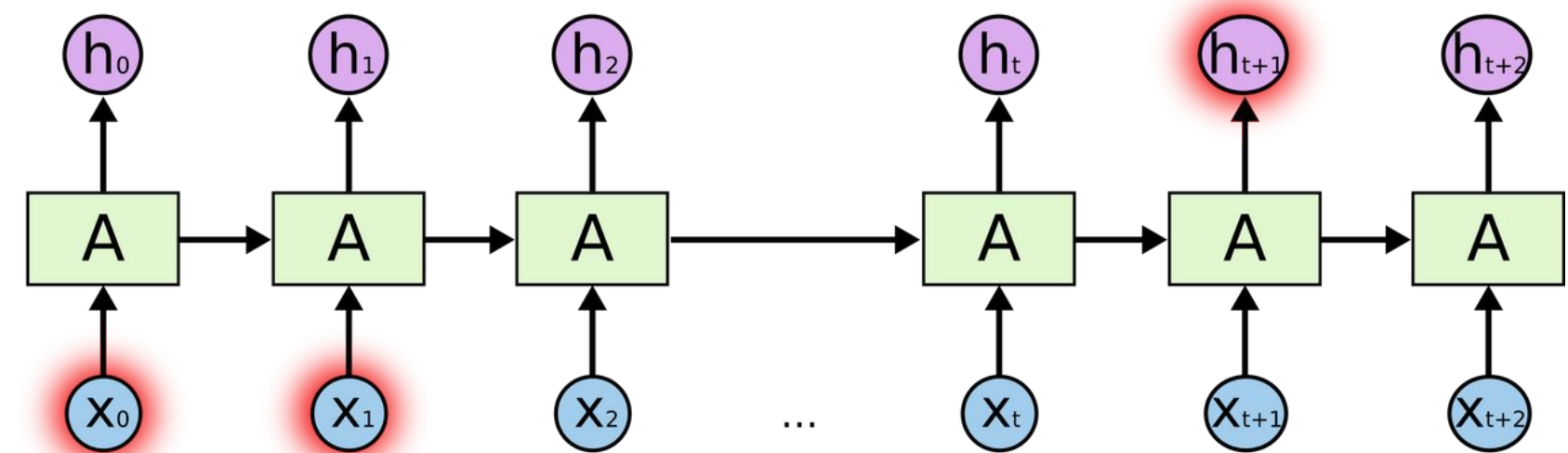


Good:

- When information is near

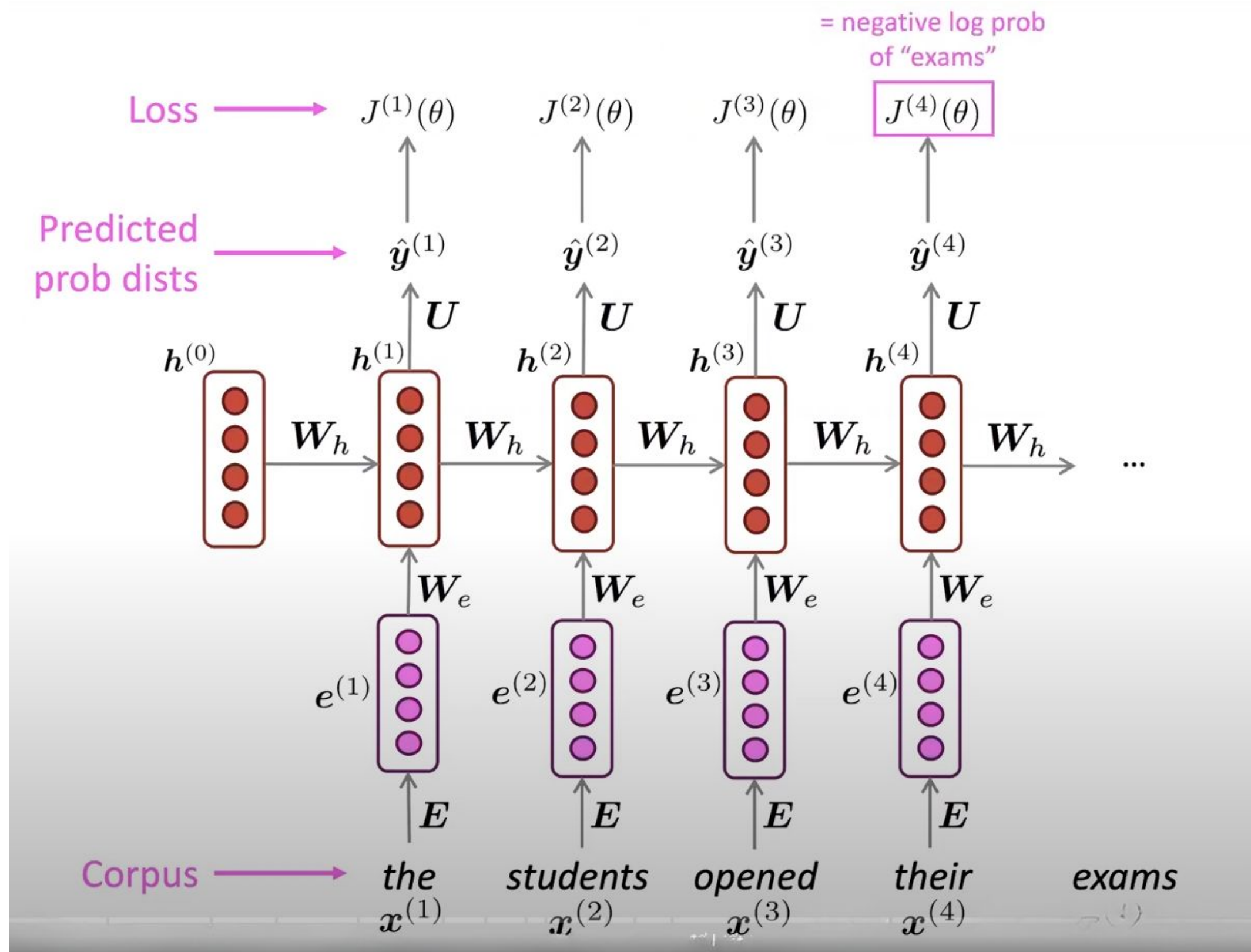
Bad:

- When information is far



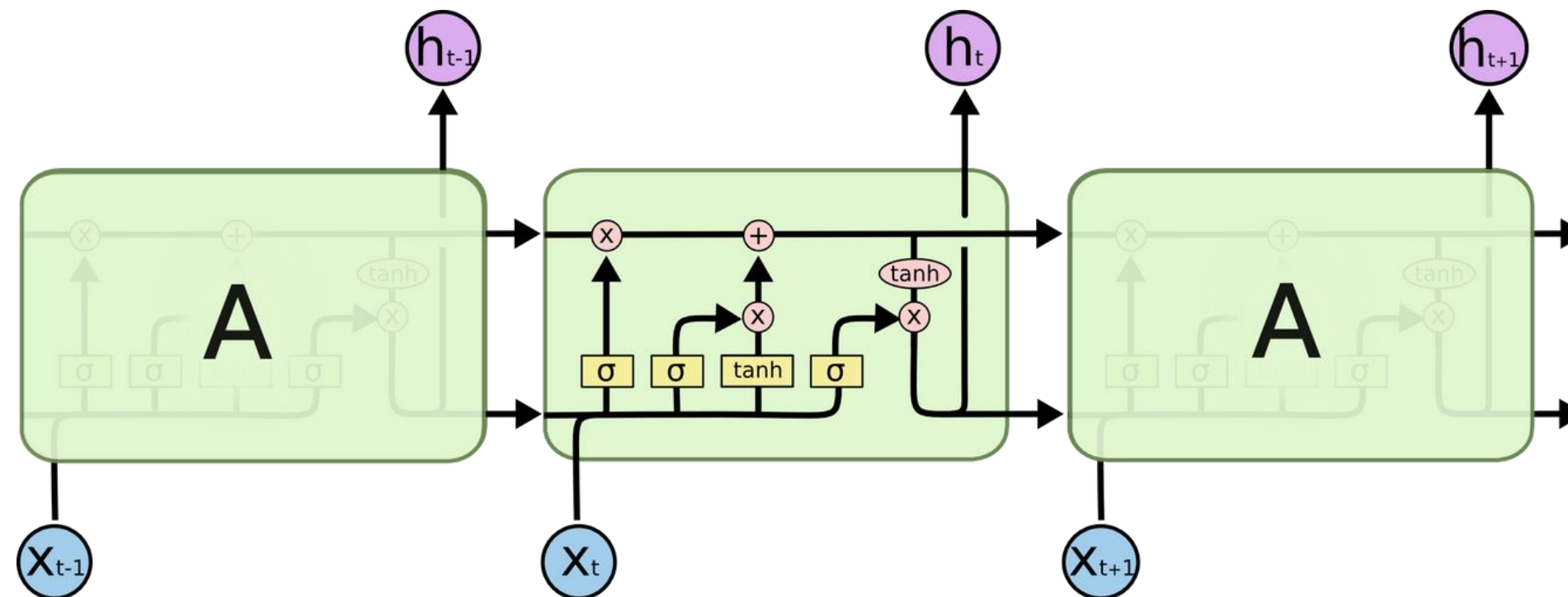
Recurrent Neural Network

Detail architecture



Long Short Term Memory

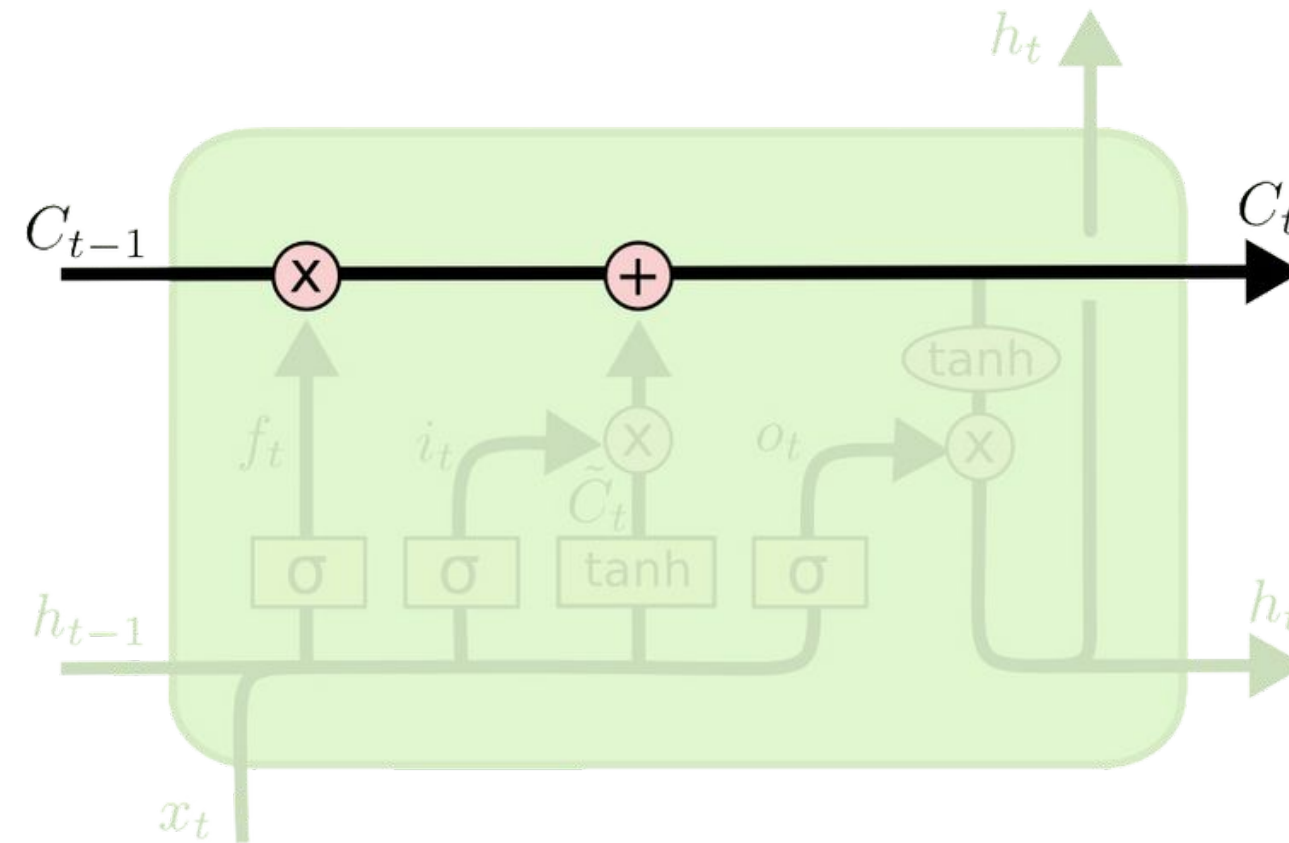
Architecture



Has very complicated structure
inside

Long Short Term Memory

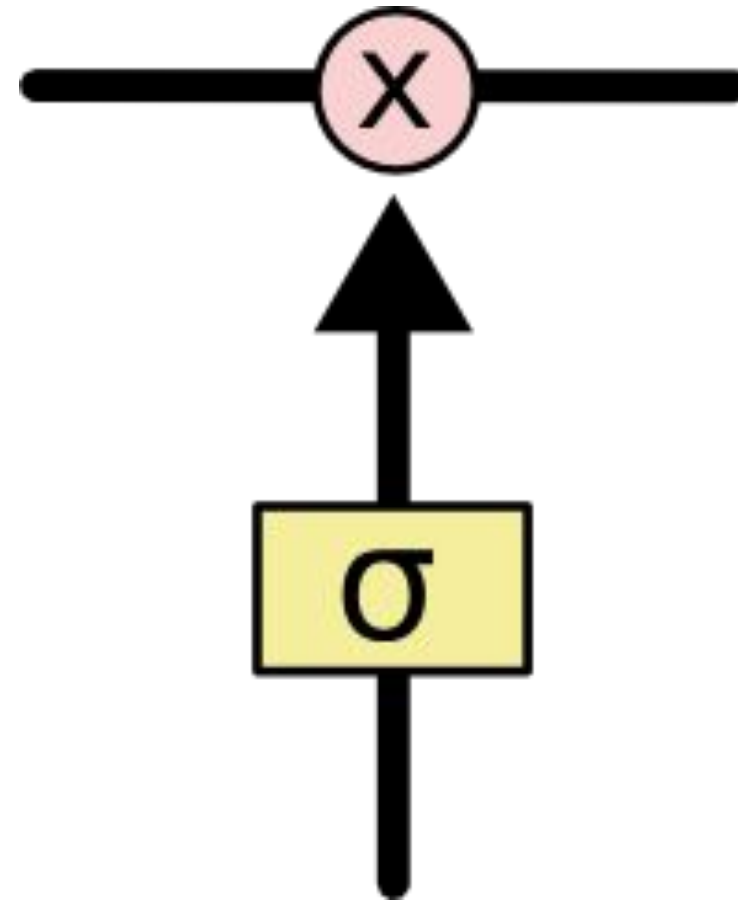
Core idea behind LSTM



- Cell state
 - Its running straight down the entire chain
 - With only some minor change
 - Easy to remember information

Long Short Term Memory

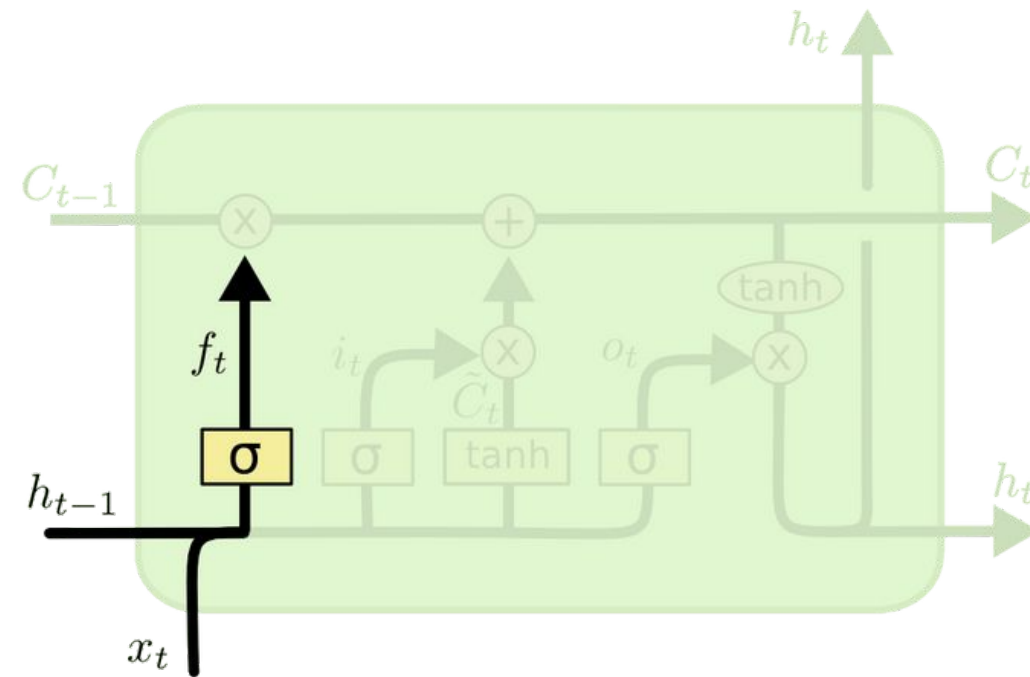
Core idea behind LSTM



- Gate
 - A way to optionally let information through
 - Describing how much of each component should be let through

Long Short Term Memory

Step by step

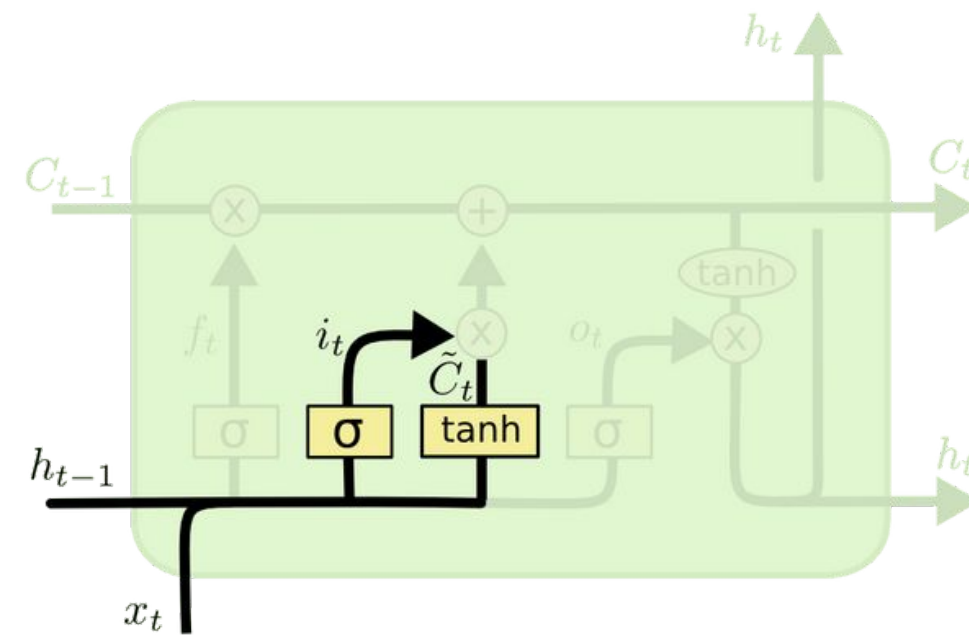


$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Forget gate
 - The first step is decide what information we need.
 - A complete 1 is keep this
 - A completely 0 is forget this
 - In between 1 and 0 is keep part of this

Long Short Term Memory

Step by step

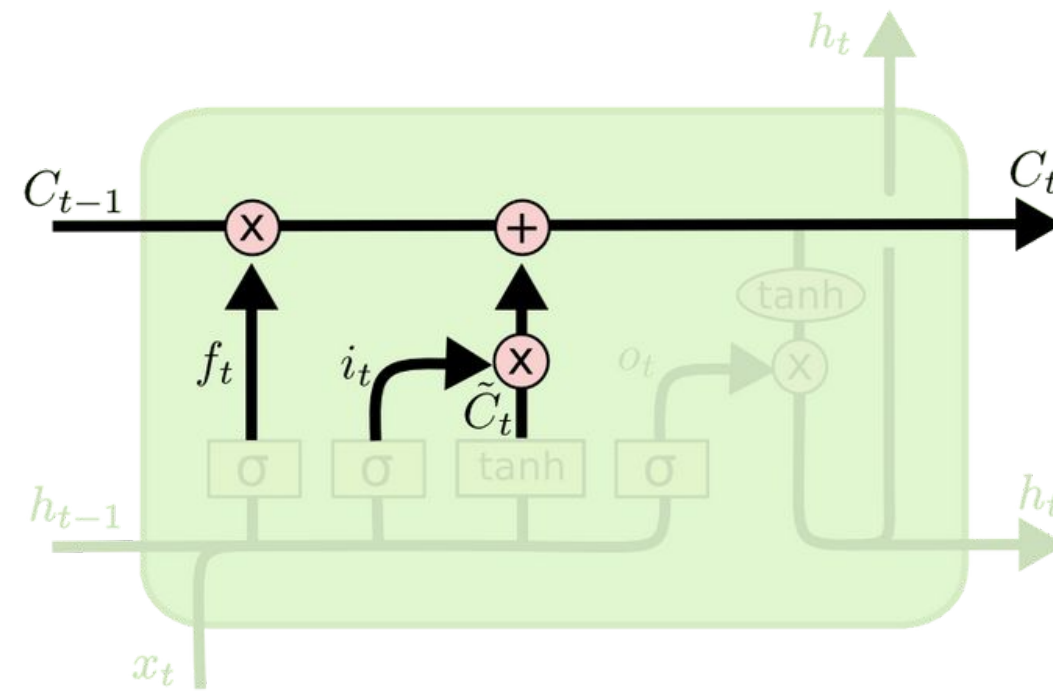


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- Input gate
 - Decide which value we will update.
 - $\tanh$ layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state.

Long Short Term Memory

Step by step

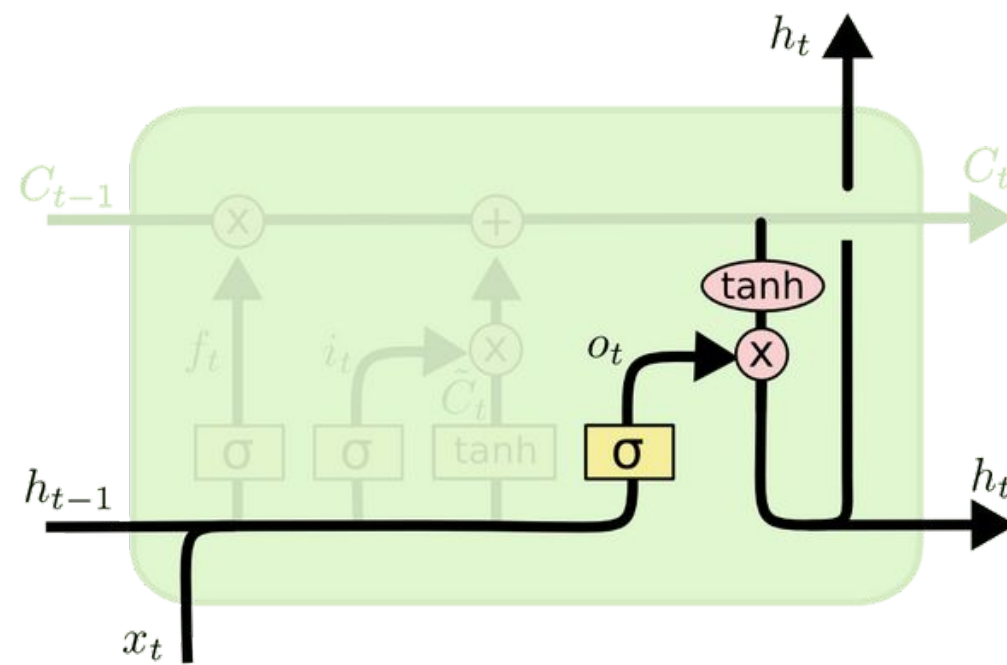


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Update cell state
 - We multiply the old state by f_t , forgetting the things we decided to forget earlier.
 - Then we add $i_t * \tilde{C}_t$.

Long Short Term Memory

Step by step



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

- Decide what we are going to output
 - Run a sigmoid layer which decides what parts of the cell state we're going to output.
 - Put the cell state through tanh.
 - Multiply it by the output of the sigmoid gate

Long Short Term Memory

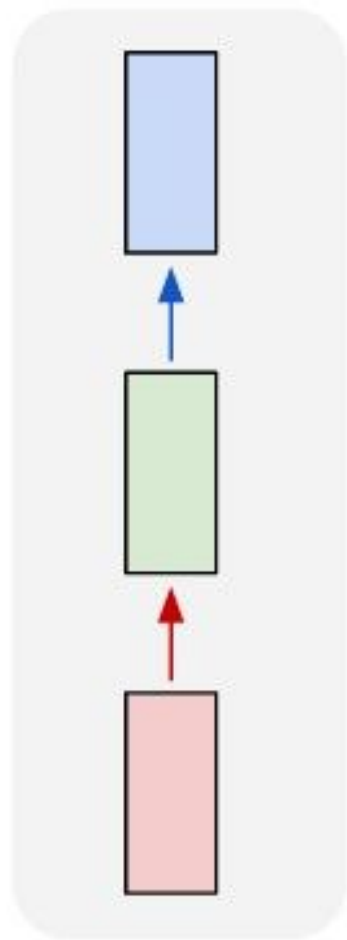
How to evaluate language model

- The standard evaluation metrics for Language Model is perplexity.
 - A metric that quantifies how uncertain a model is about the predictions it makes
 - If it's high, your model is bad
 - If it's low, your model is good

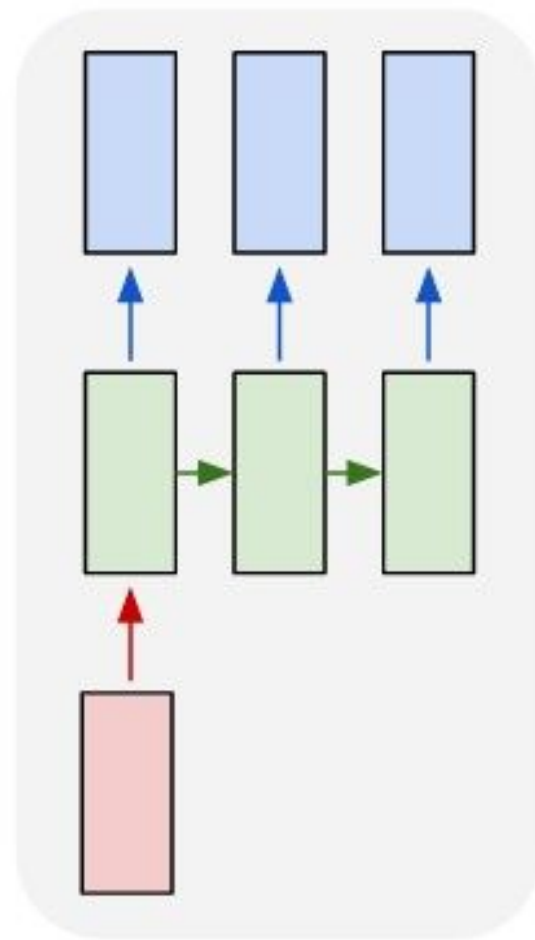
$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T}$$

Long Short Term Memory - Variations

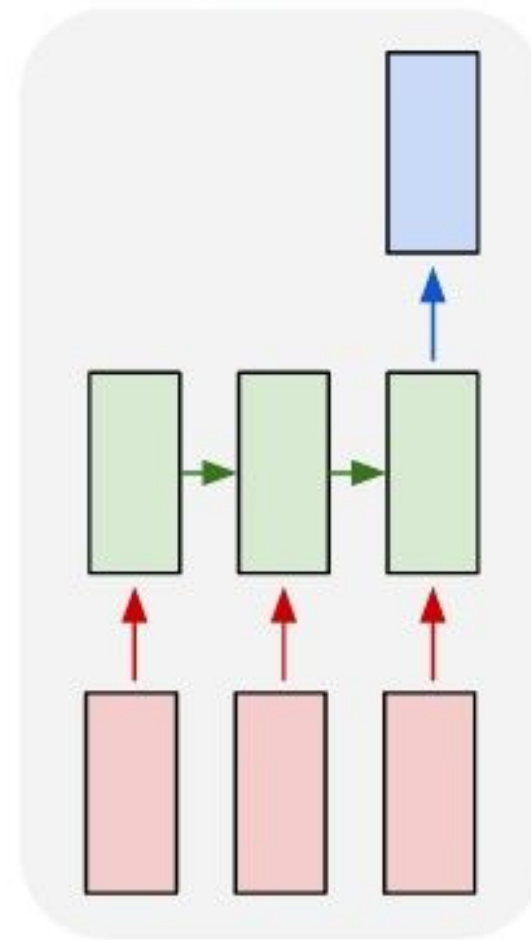
one to one



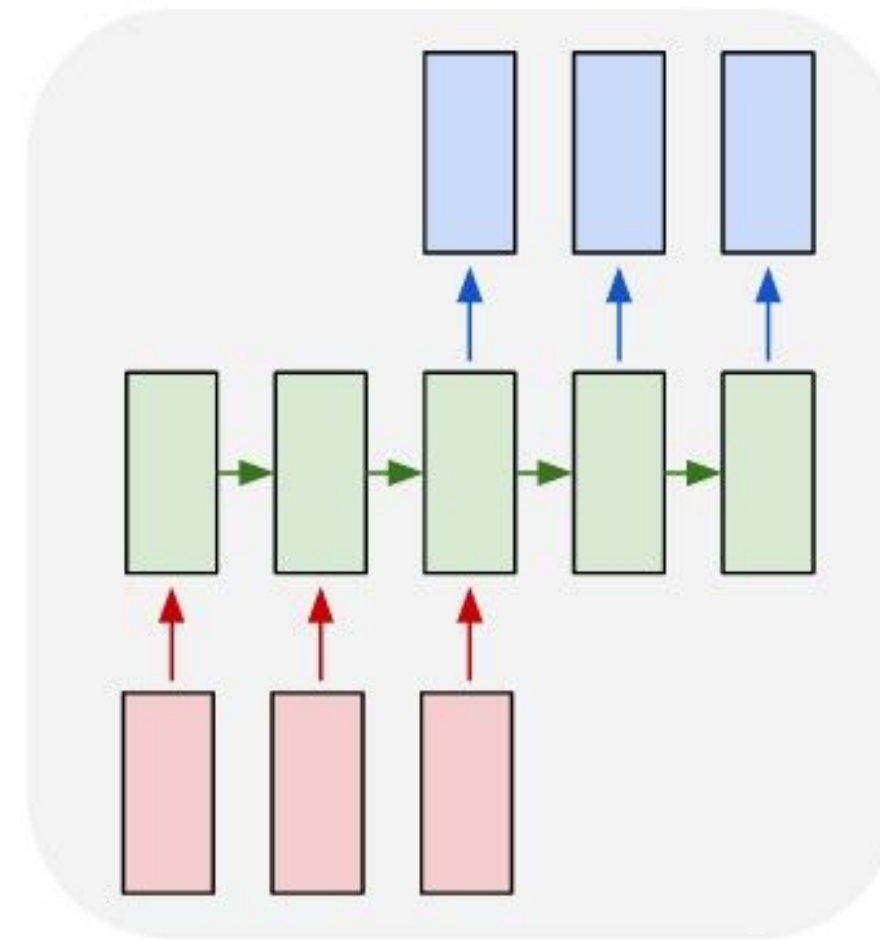
one to many



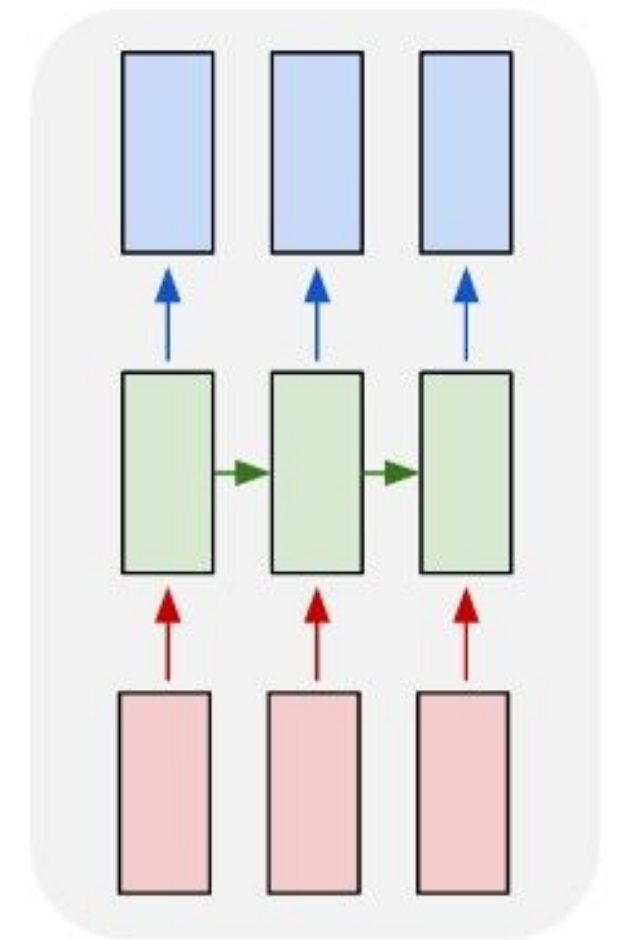
many to one



many to many

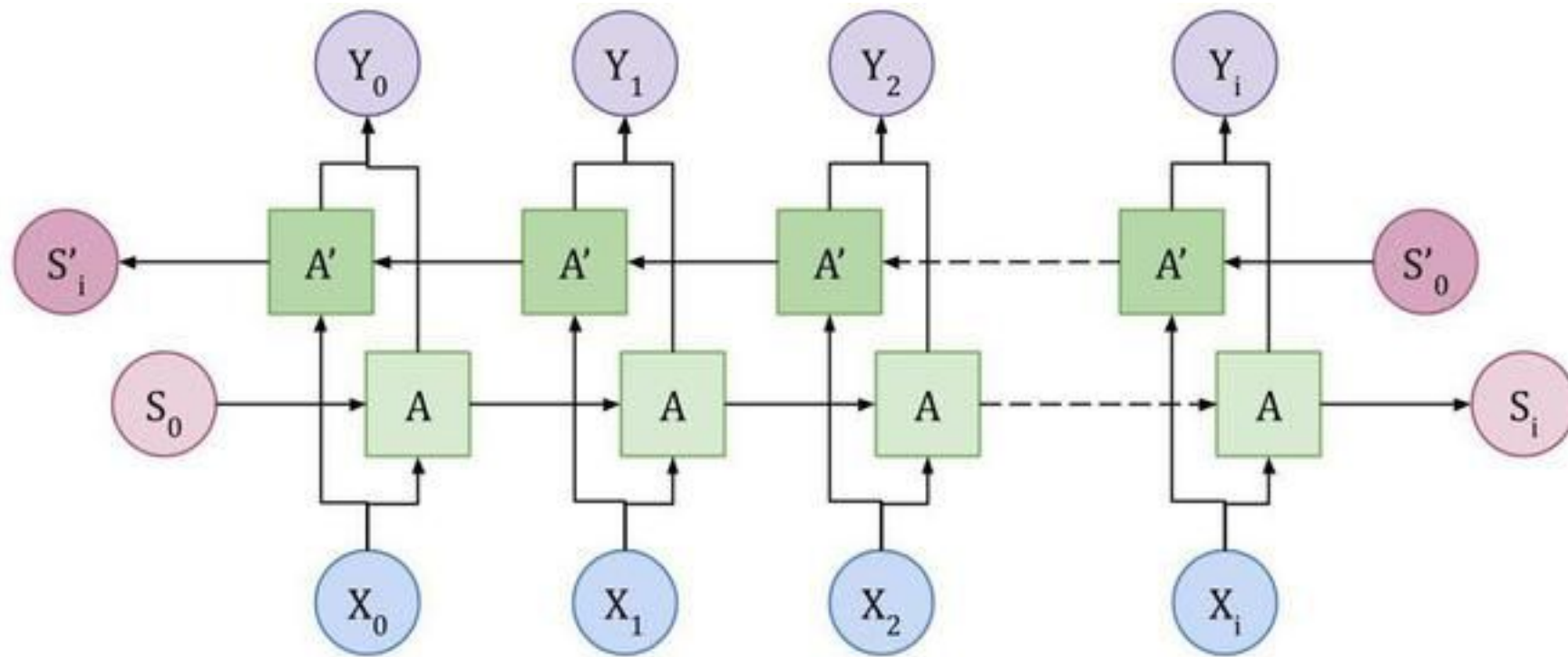


many to many

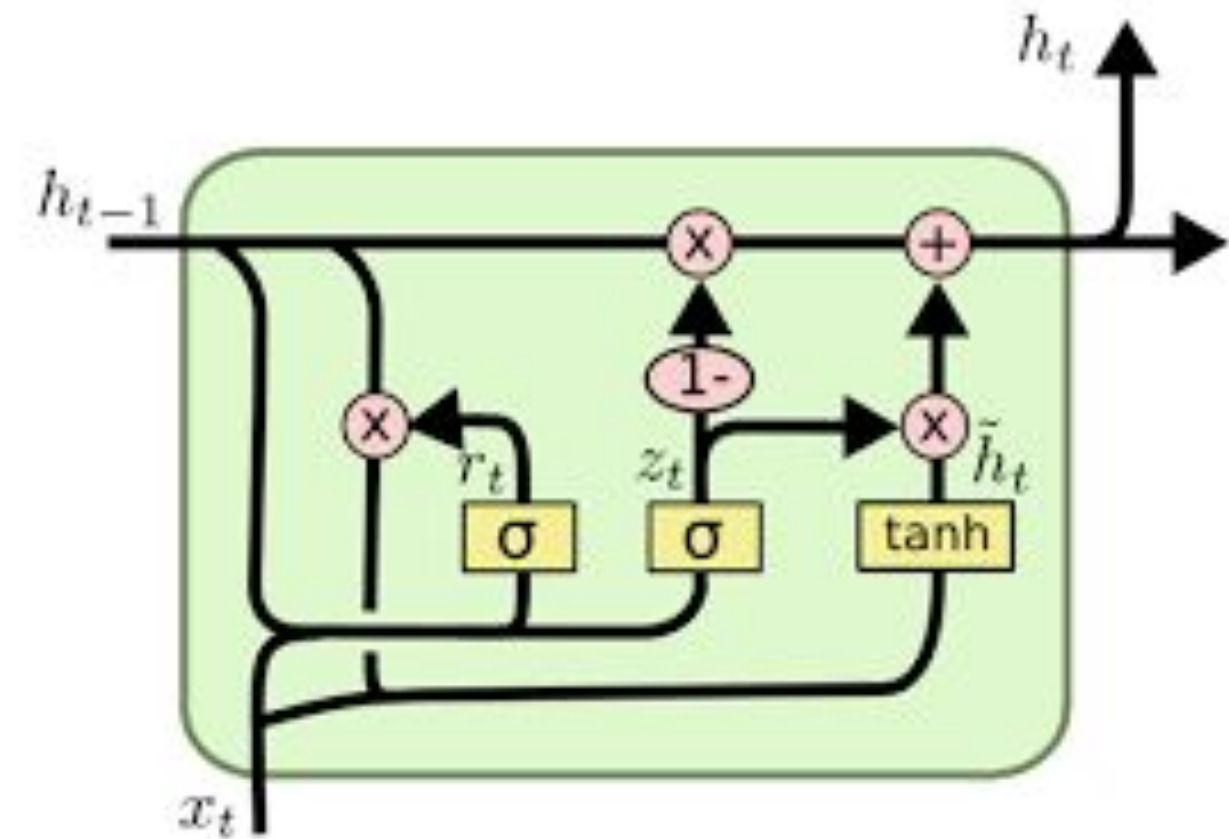


Long Short Term Memory - Variations

BiLSTM



GRU



THANK YOU!

Email: contact@tmainnovation.vn



Website: www.tmainnovation.vn

